

Privacy Leakage from a Thousand Words: Millipixel Location Recovery from Dot Maps

Yuntao Du*
Purdue University
West Lafayette, Indiana, USA
ytdu@purdue.edu

Tanishq Pauskar*
Purdue University
West Lafayette, Indiana, USA
tpauskar@purdue.edu

Hao Wang
Purdue University
West Lafayette, Indiana, USA
wang5329@purdue.edu

Jing Su
Indiana University School of Medicine
Indianapolis, Indiana, USA
su1@iu.edu

Ninghui Li
Purdue University
West Lafayette, Indiana, USA
ninghui@purdue.edu

Abstract

Dot maps, which visualize individual data points as dots over a geographic region, are widely used across diverse domains to represent spatial patterns in sensitive data. However, the understanding of the privacy risks associated with dot maps remains limited, particularly for maps covering large geographic areas. In this paper, we systematically analyze these risks and present AutoLocate, an automated framework for high-precision location recovery. At its core, AutoLocate exploits *anti-aliasing artifacts* introduced during map rendering, which inadvertently encode sub-pixel information about dot locations. AutoLocate formulates location recovery as a black-box optimization problem, iteratively refining estimated coordinates by minimizing perceptual discrepancies over these artifacts between the target map and rendered candidate maps. Extensive experiments on both real-world and synthetic datasets, across different attack scenarios and a broad range of map configurations (e.g., map scale, background, resolution), demonstrate the effectiveness of AutoLocate. In particular, it achieves average recovery errors as low as 1 meter (approximately 0.0002 pixel precision) on small-scale maps of the United States, over 200× more accurate than existing approaches. We also propose mitigation strategies and introduce a privacy risk assessment tool to help practitioners evaluate and reduce privacy leakage when publishing dot maps.

CCS Concepts

• Security and privacy;

Keywords

privacy attacks; dot maps; re-identification

ACM Reference Format:

Yuntao Du, Tanishq Pauskar, Hao Wang, Jing Su, and Ninghui Li. 2026. Privacy Leakage from a Thousand Words: Millipixel Location Recovery from Dot Maps. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026.

*Equal contribution.



This work is licensed under a Creative Commons Attribution 4.0 International License. CCS '26, The Hague, Netherlands

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
<https://doi.org/10.1145/3830454.3846524>

The Hague, Netherlands. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3830454.3846524>

1 Introduction

A well-known adage in communication is “A picture is worth a thousand words”. A commonly used type of picture is the dot map (also known as the dot distribution/density map), which employs point symbols to visualize the geographic distribution of a large number of related phenomena. Dot maps rely on visual scatter to show spatial patterns, especially variations in density. They are often used in important fields such as medical research, urban planning, and environmental studies [16, 18]. By representing each instance’s location as a “dot” (which may take the form of a circle, triangle, or other symbols) on a map, researchers can detect spatial patterns, identify clusters, and trace potential sources of outbreaks [61, 75, 77]. For instance, Soetens et al. [77] demonstrate the use of dot maps in Germany and the Netherlands by plotting individual disease cases to reveal their spatial distribution and highlight outbreak clusters. The rapid development of map visualization platforms, ranging from professional Geographic Information Systems (GIS) (e.g., ArcGIS [29] and QGIS [70]) to commercial visualization tools (e.g., Tableau [82]) and programming libraries (e.g., GeoPandas [35] and R [71]), has made it easy to obtain precise geolocation information and publish highly accurate dot maps.

While visualizations from dot maps offer clear insights into spatial relationships, they also raise significant concerns about the privacy of individuals represented on the map. This issue becomes particularly critical in privacy-sensitive domains such as healthcare, where dot maps are used for disease surveillance, risk assessment, and monitoring of public health trends [16, 66]. As highlighted in previous studies [45], the publication of raw geospatial data can introduce serious risks, including threats to personal safety from targeted crimes, legal and ethical violations due to privacy breaches, and social consequences such as neighborhood stigmatization. Despite these risks, dot maps remain a widely adopted and indispensable spatial visualization and analysis tool, with their use continuing to grow across a broad range of disciplines (see Section 2 for a detailed overview). Therefore, to balance individual privacy and utility, it is essential to develop methods that can accurately assess privacy risks when publishing dot maps.

Prior studies have investigated these risks by examining how accurately locations can be recovered from dot maps [22, 45, 90].

In these works, researchers first identify each dot’s centroid using methods such as manual visual inspection [10, 22, 53, 54] or unsupervised learning [11]. They then encode the estimated centroid to its corresponding geographic coordinates and use the resulting recovery error as a measure of privacy risk. Using these approaches, several studies [21, 53] have demonstrated that it is possible to recover individual locations from large-scale dot maps (*i.e.*, covering a limited area), with average errors around 100 meters. For instance, one study [54] re-identified residential locations with an average error of 96.38 meters from a dot map of a parish in the United States.

However, these methods fail on dot maps that cover broad geographic regions. As shown in our experiments, applying them to maps spanning countries or continents yields recovery errors on the order of hundreds of meters. To the best of our knowledge, little progress has been made on location recovery from dot maps over the past two decades. As a result, practitioners may implicitly assume that publishing maps over large areas poses limited privacy concerns. A concrete example is the U.S. Centers for Disease Control and Prevention (CDC) cartographic guideline [15], which treats privacy risks differently based on geographic scope. For maps depicting small areas, the guidelines recommend omitting locational references such as streets and landmarks, noting that “confidentiality is more likely to be ensured”. In contrast, for maps covering large areas, the guidelines assume that individual points become visually “imperceptible”, thereby permitting only a general view of geographic distribution “without enabling identification of an individual”.

In this paper, we propose a powerful location recovery attack that remains accurate *even on dot maps with broad geographic coverage*. Our key insight is that recovery accuracy can be dramatically improved by exploiting the *anti-aliasing artifacts* produced when rendering dot symbols. Anti-aliasing [34, 55] is a standard rendering technique that smooths jagged edges by blending the colors of boundary pixels according to the fraction of each pixel covered by the underlying shape (illustrated in Figure 1). While this improves visual quality, the blended colors inadvertently encode sub-pixel information about a dot’s geometric centroid, which corresponds to the exact location of the individual represented by the dot. By reverse-engineering these artifacts, we can recover dot coordinates with precision far surpassing that of previous approaches.

Building on this insight, we introduce AutoLocate, an automated location recovery framework that leverages anti-aliasing artifacts to infer precise geographic coordinates from dot maps. At the core of AutoLocate is *perceptual coordinate descent*, an iterative optimization algorithm that refines location estimates by generating candidate maps to minimize the difference in anti-aliasing artifacts between the generated and target dots. Extensive experiments on synthetic and real population data using three popular visualization tools (*i.e.*, QGIS, GeoPandas, and R) demonstrate the effectiveness and robustness of AutoLocate across a wide range of map configurations (*e.g.*, scales, backgrounds, resolutions, and formats). In particular, our experiments show that AutoLocate dramatically improves location recovery accuracy, achieving errors of **approximately 1 meter (0.0002 pixel precision)** on maps covering large regions (*e.g.*, maps of the United States), outperforming existing methods by up to **200×** in recovery accuracy.

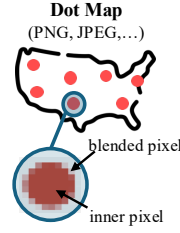


Figure 1: Anti-aliasing in dot map.

Table 1: Widespread use of raster dot maps across domains and sensitive attributes.

Application	Sensitive Attribute	Example
Public Health	Patient home addresses	[12, 33, 58, 67]
Criminology	Crime incident locations	[27, 38]
Ecology	Species habitats	[65, 68, 85]
Social Science	Demographics	[23, 51]
Education	Student locations	[4, 89]
Archaeology	Artifact find spots	[8, 40, 41]

Our work challenges the common assumption in geographic data visualization that scale alone protects privacy, and it highlights the importance of examining how maps are constructed when assessing their privacy risks. We also explore several mitigation strategies and develop a privacy risk assessment tool. This tool uses population density information to recommend a coordinate quantization level that meets a target anonymity level, helping practitioners mitigate privacy risks when publishing dot maps. In summary, we make the following contributions:

- We systematically study the privacy risks of dot maps by proposing an automated location recovery framework named AutoLocate.
- We design a new location recovery algorithm that exploits anti-aliasing artifacts in dot maps for high-precision location estimation, without requiring any knowledge of the map generation or rendering mechanism used.
- Extensive experiments show that AutoLocate is highly effective, achieving over 200× lower error than prior approaches at recovering dot locations, and remains robust across different map configurations and attack scenarios.
- We present an assessment tool to help practitioners evaluate and mitigate the privacy risks of their maps.

Roadmap. The rest of this paper is organized as follows. Section 2 provides background on dot maps and their use. Section 3 defines the threat model and attack scenarios. We then detail our location recovery framework in Section 4. Section 5 presents the experimental results of the proposed attacks. Section 6 discusses mitigation strategies and the proposed privacy risk assessment tools. Related work is detailed in Section 7, and the paper concludes in Section 8.

2 Background

Widespread Use of Dot Maps. Dot maps have long served as a fundamental tool for revealing spatial patterns and supporting decision-making across diverse disciplines. One of the most famous early examples is John Snow’s 1854 Broad Street cholera map [87], which plotted individual cholera cases as dots, enabling the visual identification of a contaminated water pump as the source of the outbreak. While this seminal work laid the foundation for modern epidemiology, the utility of dot maps today extends far beyond medical research; they are now an essential visualization method employed by government agencies, major media outlets, and researchers to communicate complex spatial data.

To demonstrate their prevalence, we surveyed recent publications and reports (see Appendix C for methodology), with Table 1

providing examples across these varied domains. We have several key observations from this survey: (i) Dot maps are widely used to visualize highly sensitive attributes, such as patient home addresses, crime incident locations, and household demographics. (ii) Despite the sensitive nature of this location data and the widespread reliance on dot maps, none of the surveyed publications explicitly document data protection practices for these visualizations. (iii) All published dot maps in Table 1 are *raster images*, representing the map as a grid of colored pixels. This dominance is a practical necessity: when organizations visualize dense populations containing thousands or millions of points, rasterization collapses massive spatial datasets into a single, fixed-resolution image. This ensures storage efficiency, platform compatibility, and suitability for print and publication.

The extensive use of raster dot maps for sensitive data highlights the critical need to systematically assess their privacy risks. Consequently, this paper focuses on analyzing the specific vulnerabilities of rasterized dot maps. As shown in Section 5, our attack achieves high recovery accuracy on dot maps across various scales and configurations, revealing significant privacy flaws in this widely adopted visualization practice.

Anti-Aliasing in Dot Maps. To render dot shapes on a discrete pixel grid, visualization tools apply spatial anti-aliasing by default [34, 55]. Anti-aliasing is a standard graphics technique designed to smooth the edges of rendered elements, mitigating the jagged artifacts (*i.e.*, aliasing) that inherently occur when approximating continuous shapes on a finite-resolution display [42]. The key idea is to blend the colors of boundary pixels with the underlying map background according to the fraction of each pixel covered by the dot shape [20, 37]. This produces intermediate color values along edges, resulting in smoother and more visually natural boundaries. Over the past decades, various algorithms have been developed to balance rendering quality and efficiency, such as Supersampling (SSAA) [73], Multisampling (MSAA) [69], and Fast Approximate Anti-Aliasing (FXAA) [59]. As a concrete example, SSAA divides each pixel into n sub-samples and determines whether the center of each sub-sample falls inside the dot or on the background. The pixel color is then computed by averaging the colors of these sub-samples:

$$I_{x,y} = \frac{m}{n} z + \left(1 - \frac{m}{n}\right) B_{x,y},$$

where z denotes the dot color, $B_{x,y}$ denotes the background color, and m is the number of sub-samples, out of n , whose centers fall inside the dot. As the dot moves by a sub-pixel amount, m changes accordingly, so the pixel color encodes the dot’s sub-pixel position.

Our key insight is that the subtle color gradients produced by this edge blending can be exploited to infer the locations of dots with greater precision, potentially down to the millipixel level. While our attack exploits these anti-aliasing artifacts, it does not depend on any specific knowledge of the anti-aliasing algorithms used to generate the target dot map.

3 Threat Model and Attack Scenarios

Adversary’s Goal. Given a raster map image $I \in \mathbb{Z}^{W \times H \times 3}$ (with width W , height H , and RGB color channels, where each channel contains integer values between 0 and 255), each dot in the image



Figure 2: Examples of three background types of dot maps.

represents the location of an individual. The adversary’s objective is to infer the underlying geographic coordinates (*i.e.*, latitude and longitude) associated with every dot.

Adversary’s Capabilities. We make the following realistic assumptions about the adversary’s capabilities:

- **Raster Image Access.** The adversary has access to the raw pixel data of the raster dot map. In practice, this requires minimal effort: dot maps published on websites or in articles can be saved directly, and maps embedded in PDF documents can be easily extracted at their original resolution using tools like `pdfimages` [2].
- **Dot Properties.** The adversary knows the visual properties of the target dots, including their geometry ϕ (*e.g.*, circle), size ρ , and color z , from which the pixel area of a single dot μ directly follows. These properties are easy to obtain by visual inspection and image editing tools (*e.g.*, a pixel selector).
- **Map Rendering Tool.** The adversary has access to a map visualization tool, modeled as a rendering function $\mathcal{R}$, that is the same as or similar to the tool used to generate the target map. We show in Section 5.3 that the attack remains effective when the adversary’s tool differs from the one used to generate the target map. We treat the rendering process as a black box: the adversary needs no knowledge of its internal mechanisms, such as the specific anti-aliasing algorithm.
- **Coordinate Transformation.** The adversary can learn a coordinate transformation function, *i.e.*, $\mathcal{F} : (x, y) \mapsto (\text{lat}, \text{lon})$, which maps coordinates (x, y) in the raster map to geographic coordinates. This transformation function can be derived from map legends (which provide scale and projection details) or reconstructed using the georeferencing features of modern GIS tools (*e.g.*, QGIS) to align the map with a known coordinate system.

Together, these capabilities enable the adversary to render new dot maps and compare them against the target map I to accurately recover the dot locations, as detailed later in our attacks.

Attack Scenarios. As discussed in Section 2, anti-aliasing blends each dot’s boundary pixels with the underlying map background $B \in \mathbb{Z}^{W \times H \times 3}$, *i.e.*, the base map onto which the dots are rendered. Therefore, knowledge of the background provides additional signals for recovering dot locations. Published dot maps adopt backgrounds of varying complexity, ranging from a blank canvas to a street map or a satellite image, as demonstrated in Figure 2. We distinguish two attack scenarios based on whether the background is available to the adversary:

- **Background Known.** The adversary has access to the map background B used to generate the dot map I . This is realistic even when the original background is not directly provided: many published dot maps (as shown in Table 1) use simple uniform

Algorithm 1 Baseline: PixelMatch. The algorithm identifies connected components of pixels matching the target dot color and computes the centroid of each as the estimated location.

Require: Target dot map I , dot color z

- 1: $\mathcal{P} \leftarrow \text{FindConnectedComponents}(I, z)$
- 2: $C \leftarrow \emptyset$ ▷ initialize set of estimated locations
- 3: **for** each component $P \in \mathcal{P}$ **do**
- 4: $(x_c, y_c) \leftarrow \frac{1}{|P|} \sum_{(x,y) \in P} (x, y)$
- 5: $C \leftarrow C \cup \{(x_c, y_c)\}$
- 6: **end for**
- 7: **return** C

colors (e.g., white) or standard basemaps from public repositories (e.g., OpenStreetMap [19]) that are easy to replicate.

- *Background Unknown.* The adversary only has access to the final map I . This occurs when the map employs a proprietary or custom background that is not publicly available. In this case, the adversary must infer dot locations solely from the target map.

Although some map exports include auxiliary data (e.g., metadata in TIFF), our attack does not rely on such information. This ensures that our approach remains effective across common image formats, as shown in Section 5.2.

Graph Representation of Raster Maps. We model the raster map I as a grid graph in which each pixel corresponds to a node, and edges connect each node to its eight spatial neighbors (*i.e.*, the horizontally, vertically, and diagonally adjacent pixels). A *dot region* is then defined as a connected component of the subgraph induced by the pixels whose color matches the dot color z , which can be efficiently identified using standard graph traversal algorithms (e.g., breadth-first search). This formulation provides a notion of pixel adjacency that underpins the operations of our attack.

Baseline: PixelMatch. A straightforward method for estimating dot locations is to compute the centroid of each dot region, which we call PixelMatch. As detailed in Algorithm 1, the algorithm first extracts all connected components composed of pixels matching the target color z (Line 1). Each connected component P corresponds to a candidate dot, and the algorithm computes the mean coordinate of its constituent pixels to determine its centroid (Line 4). Finally, these centroids are mapped to geographic coordinates using the transformation $\mathcal{F}$ to obtain the estimated locations.

Missed Opportunities of Existing Approaches. Prior studies [10, 11, 54] have used the idea of PixelMatch, either through manual inspection or by using GIS tools for centroid estimation. However, they fail to fully exploit the available information and do not account for realistic scenarios in which dots may overlap:

- *Anti-aliasing Artifacts on Dot Boundaries.* PixelMatch estimates a dot’s location using only its inner pixels, *i.e.*, the pixels whose color exactly matches the dot color. The set of inner pixels changes only when the dot moves far enough to fully cover or uncover a pixel. A range of sub-pixel dot locations therefore produces the same set of inner pixels, from which PixelMatch would estimate the same dot centroid. Anti-aliased boundary pixels provide the additional information needed to distinguish these locations, as their color values are sensitive to the dot’s sub-pixel position.

However, existing approaches overlook these boundary pixels because their color does not exactly match the dot color. Moreover, as demonstrated in Section 5, simply incorporating boundary pixels into the centroid computation is still ineffective for high-precision location recovery.

- *Overlapping Dots.* In dense regions, multiple dots may overlap, forming overlapping dot regions where the boundaries of individual dots merge. PixelMatch treats this merged component as one dot, computing a single centroid and failing to recover the individual dot locations within.

4 AutoLocate: A Framework for Automated Location Recovery from Dot Maps

In this section, we present AutoLocate, an automated framework for high-precision location recovery from dot maps. AutoLocate formulates location recovery as an optimization problem: it iteratively adjusts the estimated dot locations to minimize the perceptual discrepancy between the target map and a rendered candidate map, using anti-aliasing artifacts as the optimization signal.

4.1 Attack Method

We first consider the attack scenario where the adversary possesses the map background B . In this setting, the adversary can render new dot maps from a set of estimated coordinates and compare them with the target map. By analyzing the discrepancies at anti-aliased boundaries, the adversary iteratively adjusts the coordinates until the rendered map closely matches the target. We first define the different types of pixels in dots, formulate the optimization loss function, discuss the handling of overlapping dots, and present the optimization algorithm and the complete framework.

Inner Pixels and Boundary Pixels. For each dot, we identify its *dot region* as the connected component of pixels matching the dot color z that contains the dot’s estimated location. The pixels in this connected component are the *inner pixels* of the dot: they are fully covered by the dot color and carry no sub-pixel positional information. The *boundary pixels* are pixels that (i) are four-neighbors (*i.e.*, up, down, left, or right) of at least one inner pixel, and (ii) do not belong to the connected component, *i.e.*, their color differs from z . The detailed procedure for identifying these pixels is in Algorithm 3. These pixels are anti-aliasing artifacts produced by blending the dot color with the underlying map background, and their color values are sensitive to the dot’s sub-pixel position. We denote the set of boundary pixels for a dot as $\mathcal{S}$.

Perceptual Loss Function. Given a set of estimated dot locations C , the dot’s properties (color z , shape ϕ , size ρ), and the background B , the adversary renders a candidate dot map I' using the map rendering function $\mathcal{R}$:

$$I' \leftarrow \mathcal{R}(B, C, z, \phi, \rho).$$

A good location estimate should produce a candidate map that perceptually matches the target map I , particularly at the boundary pixels $\mathcal{S}$ where small positional shifts produce measurable color changes. We first define the perceptual discrepancy $d(\cdot, \cdot)$ between two pixels a and b as the L_1 distance between their RGB channels:

$$d(a, b) = |a^r - b^r| + |a^g - b^g| + |a^b - b^b|.$$

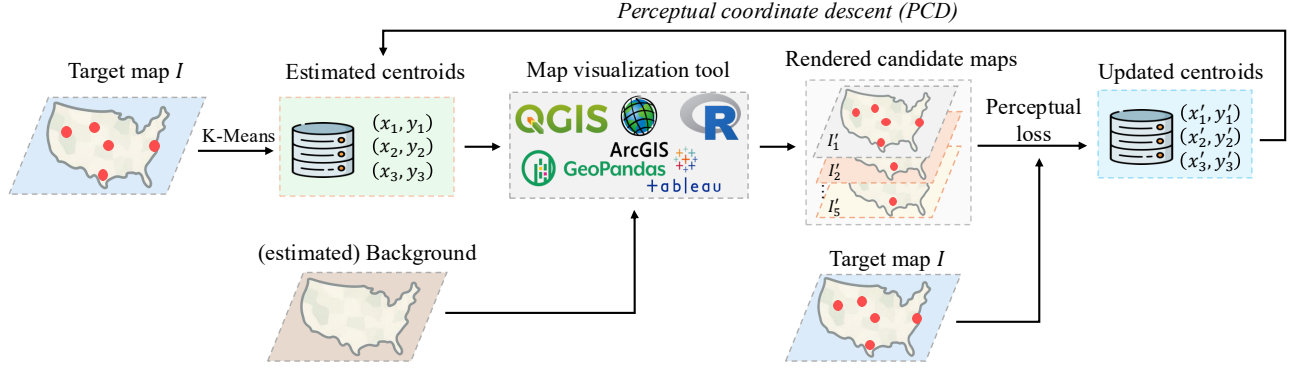


Figure 3: Illustration of the key processes in AutoLocate. The adversary first initializes dot locations using K-Means clustering. Using map visualization tools and the (estimated) map background, the adversary renders candidate maps for five search directions (i.e., stay, left, right, up, and down). The direction that minimizes the perceptual loss is selected for each dot. This process is repeated to refine the location estimates. Best viewed in color.

A boundary pixel at position (x, y) is produced by anti-aliasing, which blends the dot color z with the background color $B_{x,y}$; consequently, its color lies between these two values. We define the relative color deviation of the pixel from the background, normalized by the maximum possible deviation $d(z, B_{x,y})$:

$$\delta_{x,y} = \frac{d(I_{x,y}, B_{x,y})}{d(z, B_{x,y})}.$$

Using the same approach, we compute the corresponding relative deviation $\delta'_{x,y}$ for the candidate map I' . For each dot $c \in C$, we calculate the total perceptual loss over its boundary pixels S by summing the absolute differences between these relative deviations:

$$\mathcal{L}(I, I', B, S) = \sum_{(x,y) \in S} |\delta_{x,y} - \delta'_{x,y}|.$$

This focuses optimization on the boundary where anti-aliasing encodes fine-grained positional information.

Handling Overlapping Dots. The above definitions assume each dot occupies its own connected component. In dense regions, multiple dots may overlap, causing their connected components to merge into a single component. This raises two challenges: the number of individual dots within the merged component is unknown, and boundary pixels between overlapping dots may be absorbed as inner pixels. We address this through a two-stage strategy:

- **Estimating the Number of Dots.** For each connected component P , we estimate the number of overlapping dots k by dividing the total pixel area of the region by the area of a single dot (μ). We then apply K-Means clustering [64] to the pixels in P , partitioning it into k groups and yielding k initial dot locations.
- **Boundary Pixel Assignment.** To identify the relevant boundary pixels for each individual dot, we assign each boundary pixel to its nearest estimated dot centroid. This ensures that every boundary pixel belongs exclusively to a single dot, preventing contamination from neighboring overlapping dots.

This procedure enables accurate identification of boundaries for overlapping dots, providing a stable signal for exploiting anti-aliasing artifacts in location estimation.

Perceptual Coordinate Descent (PCD). While we can compute the perceptual loss for each dot's current estimated location, applying standard gradient-based optimization (e.g., SGD [50, 72]) to minimize this loss is infeasible because the rendering function $\mathcal{R}$ is a black box whose gradients are intractable to compute. To address this, we propose Perceptual Coordinate Descent (PCD), a gradient-free algorithm inspired by zeroth-order optimization [47, 49, 78]. Instead of computing gradients, PCD probes neighboring positions on the two-dimensional pixel grid and selects the move that most reduces the loss, using the boundary pixels S as the optimization signals. This process repeats until no direction produces further improvement or a maximum number of iterations is reached.

Framework Overview. The complete location recovery framework is outlined in Algorithm 2 and Figure 3. The algorithm consists of two phases. Phase 1 (lines 1–8) initializes dot locations. It extracts all connected components of pixels matching the dot color z . For each component P , it estimates the number of dots k and applies K-Means clustering to establish initial coordinates. Phase 2 (lines 9–28) iteratively refines locations using PCD. In each iteration, the algorithm considers five candidate directions (i.e., stay, left, right, up, and down) with step size η , renders a candidate map for each direction (lines 16–19), and evaluates the perceptual loss at each dot's boundary pixels. The location yielding the smallest loss is selected for each dot (lines 22–25). This process is repeated for T iterations to ensure convergence.

Note that the algorithm renders one map per search direction, in which all dots are shifted simultaneously. We also implement a per-dot alternative, where the loss for each dot is computed by rendering a separate candidate map that shifts only that dot. We compare the attack performance and efficiency of these two implementations in Appendix D.

Attack with Unknown Background. When the adversary does not possess the map background B , the optimization approach cannot be directly applied. To address this, we adapt the framework by estimating the background color of boundary pixels from the target map. Specifically, we first use the same boundary detection procedure (Algorithm 3) to identify the boundary pixels S for each

Algorithm 2 Automated Location Recovery Framework. The algorithm applies perceptual coordinate descent (detailed in Section 4) to iteratively refine initial dot locations by minimizing the perceptual loss computed on anti-aliased boundary pixels $\mathcal{S}$.

Require: Target dot map $\mathbf{I}$, (estimated) background $\mathbf{B}$, map rendering function $\mathcal{R}$, pixel area of a single dot μ , dot color $\mathbf{z}$, dot geometry ϕ , dot size ρ , iterations T , step size η

```

1: # Phase 1: Initialize dot location estimates
2:  $\mathcal{P} \leftarrow \text{FindConnectedComponents}(\mathbf{I}, \mathbf{z})$ 
3:  $\mathcal{C} \leftarrow \emptyset$  ▷ initialize location
4: for each component  $P \in \mathcal{P}$  do
5:    $k = \lceil \frac{|P|}{\mu} \rceil$  ▷ estimate # dots in the component
6:    $\mathcal{C}_P \leftarrow \text{K-Means}(P, k)$ 
7:    $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}_P$ 
8: end for

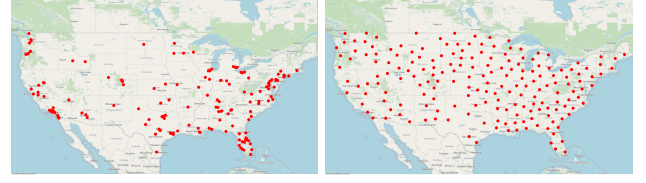
9: # Phase 2: Perceptual coordinate descent (PCD)
10: # define search directions (stay, left, right, up, down)
11:  $\mathcal{D} \leftarrow \{(0, 0), (\eta, 0), (-\eta, 0), (0, \eta), (0, -\eta)\}$ 
12: # identify boundary pixels
13:  $\{\mathcal{S}_i\}_{i=1}^{|\mathcal{C}|} \leftarrow \text{FindBoundaryPixels}(\mathcal{P}, \mathcal{C})$ 
14: for  $T$  times do
15:   # render candidate maps for each direction
16:   for each  $\mathbf{d} \in \mathcal{D}$  do
17:      $\mathcal{C}_d \leftarrow \{\mathbf{c} + \mathbf{d} \mid \mathbf{c} \in \mathcal{C}\}$  ▷ shift all locations by  $\mathbf{d}$ 
18:      $\mathbf{I}_d \leftarrow \mathcal{R}(\mathbf{B}, \mathcal{C}_d, \mathbf{z}, \phi, \rho)$  ▷ render candidate map
19:   end for
20:    $\mathcal{C}_{\text{new}} \leftarrow \emptyset$  ▷ initialize updated locations
21:   # evaluate loss for each dot across candidate maps
22:   for each dot  $\mathbf{c}_i \in \mathcal{C}$  do
23:      $\mathbf{c}^* \leftarrow \mathbf{c}_i + \arg\min_{\mathbf{d} \in \mathcal{D}} \mathcal{L}(\mathbf{I}, \mathbf{I}_d, \mathbf{B}, \mathcal{S}_i)$ 
24:      $\mathcal{C}_{\text{new}} \leftarrow \mathcal{C}_{\text{new}} \cup \{\mathbf{c}^*\}$ 
25:   end for
26:    $\mathcal{C} \leftarrow \mathcal{C}_{\text{new}}$  ▷ update all dot locations
27: end for
28: return  $\mathcal{C}$ 

```

dot. For each boundary pixel $(x, y) \in \mathcal{S}$, we estimate its local background color $\hat{\mathbf{b}}_{x,y}$ by averaging the colors of its neighboring pixels that lie outside the dot region:

$$\hat{\mathbf{b}}_{x,y} = \frac{1}{|\mathcal{N}_{x,y}|} \sum_{(u,v) \in \mathcal{N}_{x,y}} \mathbf{I}_{u,v},$$

where $\mathcal{N}_{x,y}$ denotes the eight neighbors of (x, y) that are not part of the dot region (*i.e.*, they are neither inner pixels nor boundary pixels). We then construct an estimated background $\hat{\mathbf{B}}$ by copying $\mathbf{I}$ and replacing the color of every pixel in $\mathcal{S}$ with $\hat{\mathbf{b}}_{x,y}$. Note that we do not need to estimate the background for the inner pixels; they are fully covered by the dot color $\mathbf{z}$ during rendering, so their underlying background does not influence the anti-aliasing artifacts and thus does not contribute to location estimation. The estimated background $\hat{\mathbf{B}}$ then serves as the input to Algorithm 2, and the rest of the recovery process proceeds unchanged.



(a) OpenAddresses

(b) Synthetic

Figure 4: Dot maps at the small scale (*i.e.*, United States) on the OpenAddresses and synthetic datasets.

Table 2: Overview of the evaluation coordinate datasets.

Dataset	# Dot Clusters by Overlap					Precision	Type
	1 (isolated)	2	3	4	5		
OpenAddresses	90	24	21	12	5	7 decimals	Real-world
Synthetic	188	25	25	25	25	6 decimals	Synthetic

5 Evaluation

We conduct a comprehensive evaluation of AutoLocate across various attack settings to assess the privacy risks associated with different types of dot maps. Specifically, we aim to answer the following research questions:

- **RQ1:** How effective is AutoLocate compared to existing methods across different configurations of dot maps?
- **RQ2:** How do the different components of AutoLocate impact location recovery performance? How efficient is our approach?
- **RQ3:** How does recovery accuracy vary across dots, and what factors contribute to these variations?

5.1 Experimental Setup

Evaluation Datasets. To the best of our knowledge, no public datasets or benchmarks exist for the dot map location recovery task. Moreover, using dot maps from existing publications would raise privacy concerns and lack ground truth. To address these issues, we construct two datasets of geographic coordinates for evaluation:

- **OpenAddresses.** We randomly sample coordinates from OpenAddresses [1], a free and open global collection of geocoded street addresses. These coordinates reflect real-world spatial patterns, such as variations in population and urban-rural densities, providing a representative evaluation of privacy risks in practice.
- **Synthetic.** We uniformly sample coordinates within the geographic range of each map configuration. To construct overlapping dots, we randomly perturb the sampled coordinates within a small radius to form clusters of 2, 3, 4, and 5 overlapping dots.

Each dataset consists of geographic coordinates (*i.e.*, latitude, longitude) that serve as ground-truth locations for the dots. The OpenAddresses dataset provides coordinates with seven decimal places of latitude and longitude, whereas the synthetic dataset uses six decimal places. One unit in the last decimal place corresponds to roughly 1 cm at seven decimals and 11 cm at six decimals. This level of precision is common in GPS collection systems, and standard geocoding services (*e.g.*, Google Maps [36]) provide six-decimal

Table 3: Location recovery error across different map scales. We denote our method variants as AutoLocate_{BK} (i.e., Background Known) and AutoLocate_{BU} (i.e., Background Unknown). The best result is in bold.

Map Scale	Method	OpenAddresses				Synthetic			
		Lat. Error	Lon. Error	Dist. Error (m)	Rel. Px. Error	Lat. Error	Lon. Error	Dist. Error (m)	Rel. Px. Error
Small-scale (1:10M)	PixelMatch	0.002048	0.002072	287.08 $\pm$ 105.15	0.1063 $\pm$.0389	0.001828	0.001849	256.18 $\pm$ 97.74	0.0949 $\pm$.0362
	PixelAvg	0.001685	0.001704	236.11 $\pm$ 82.69	0.0874 $\pm$.0306	0.001617	0.001638	226.44 $\pm$ 78.30	0.0839 $\pm$.0290
	Raster2Vec	0.002048	0.002072	287.08 $\pm$ 105.15	0.1063 $\pm$.0389	0.001828	0.001849	256.18 $\pm$ 97.74	0.0949 $\pm$.0362
	AutoLocate _{BU}	0.000020	0.000020	1.81 $\pm$.66	0.0007 $\pm$.0003	0.000012	0.000012	1.69 $\pm$.51	0.0006 $\pm$.0002
	AutoLocate _{BK}	0.000012	0.000012	0.95 $\pm$.64	0.0003 $\pm$.0002	0.000005	0.000006	0.88 $\pm$.52	0.0002 $\pm$.0001
Medium-scale (1:1M)	PixelMatch	0.000219	0.000223	30.65 $\pm$ 12.26	0.1135 $\pm$.0454	0.000274	0.000279	38.28 $\pm$ 10.12	0.1418 $\pm$.0375
	PixelAvg	0.000181	0.000185	25.41 $\pm$ 9.78	0.0941 $\pm$.0362	0.000243	0.000247	33.96 $\pm$ 8.18	0.1258 $\pm$.0303
	Raster2Vec	0.000219	0.000223	30.65 $\pm$ 12.26	0.1135 $\pm$.0454	0.000274	0.000279	38.28 $\pm$ 10.12	0.1418 $\pm$.0375
	AutoLocate _{BU}	0.000002	0.000002	0.25 $\pm$.06	0.0009 $\pm$.0002	0.000002	0.000002	0.21 $\pm$.06	0.0008 $\pm$.0002
	AutoLocate _{BK}	0.000001	0.000001	0.12 $\pm$.06	0.0004 $\pm$.0002	0.000001	0.000001	0.10 $\pm$.05	0.0004 $\pm$.0002
Large-scale (1:100K)	PixelMatch	0.000021	0.000022	2.99 $\pm$ 1.24	0.1107 $\pm$.0460	0.000023	0.000023	3.18 $\pm$ 1.23	0.1178 $\pm$.0454
	PixelAvg	0.000019	0.000019	2.65 $\pm$ 1.12	0.0981 $\pm$.0415	0.000017	0.000018	2.44 $\pm$.97	0.0904 $\pm$.0358
	Raster2Vec	0.000021	0.000022	2.99 $\pm$ 1.24	0.1107 $\pm$.0460	0.000023	0.000023	3.18 $\pm$ 1.23	0.1178 $\pm$.0454
	AutoLocate _{BU}	0.000001	0.000001	0.05 $\pm$.0046	0.0019 $\pm$.0002	0.000001	0.000001	0.05 $\pm$.0160	0.0019 $\pm$.0006
	AutoLocate _{BK}	0.000001	0.000001	0.05 $\pm$.0052	0.0019 $\pm$.0002	0.000001	0.000001	0.03 $\pm$.0180	0.0011 $\pm$.0007

precision by default. Moreover, using this precision ensures that errors in location recovery are attributed to the attack algorithm rather than the coarseness of the input data. Dataset statistics are summarized in Table 2, and example dot maps generated from these datasets are shown in Figure 4. We further analyze the impact of coordinate precision on attack performance in Section 5.3.

Map Configurations. To ensure our evaluation covers a realistic range of visualization practices, we systematically vary *seven* map configuration dimensions, summarized in Table 21. Specifically, we generate target maps across three geographic scales: small-scale (i.e., the United States), medium-scale (i.e., Ohio), and large-scale (i.e., Austin, Texas). For each scale, we evaluate three background types: a blank white canvas, a standard street map from OpenStreetMap [19], and a satellite map provided by Esri [30]. Maps are exported at three resolutions (96, 192, and 384 DPI), in three image formats (PNG, JPEG, and TIFF), and with three dot geometries (circle, pentagon, and triangle) at varying sizes (1, 2, and 3 mm). We use three widely adopted map visualization platforms to generate target dot maps: GeoPandas [35], QGIS [70], and R [71] (with the maps package [7]) using their default map composition settings. A demonstration of the generated small-scale maps across different backgrounds is shown in Figure 2.

Baselines and Attack Variants. In addition to the PixelMatch baseline introduced in Algorithm 1, we compare against the following location recovery algorithms:

- *PixelAvg.* This method computes the mean location of both the inner pixels (i.e., pixels matching the dot color) and the boundary pixels (i.e., pixels adjacent to the inner pixels) to estimate each dot’s centroid.
- *Raster2Vec.* Image vectorization recovers vector representations from raster images and has been widely studied in computer graphics [44, 83]. As a representative approach, we use QGIS’s built-in raster-to-vector conversion tool, which has been applied

in prior work [11] for location recovery. The dot’s location is determined as the centroid of the resulting vector geometry.

We evaluate both variants of our method: AutoLocate_{BK}, which uses perceptual coordinate descent with access to the map background, and AutoLocate_{BU}, which estimates the background from the target map when the map background is unavailable. Existing approaches focus on isolated dots and are designed specifically for that setting. Accordingly, we restrict our comparison with these baselines to the recovery of locations from isolated dots.

Evaluation Metrics. We assess the performance of location recovery algorithms by computing the median recovery error across all dots. We report this error using two metrics:

- *Absolute Geographical Error.* For each attack, we estimate the dot’s centroid in pixel coordinates and convert it to geographic coordinates (latitude and longitude). We then compute the difference between the estimated and ground-truth locations, reporting latitude error, longitude error, and the geodesic recovery error (Euclidean L_2 distance) in meters.
- *Relative Pixel Error.* We calculate the geographical error (in meters) and normalize it by the real-world distance that a single pixel represents at that map’s scale. This provides a relative error at the pixel level.

Hyperparameter Settings. We use consistent hyperparameter settings across all experiments for AutoLocate to demonstrate its robustness. Specifically, we set the number of iterations to $T = 30$ and the step size to $\eta = 0.05$ pixels, with learning rate decay [46] set to 0.75. To estimate the per-dot pixel area μ , we randomly select five isolated dots, identify their connected components, and compute the average number of pixels within these components. For rendering candidate maps, we set the map rendering function $\mathcal{R}$ to the same platform used to generate the target map. In Section 5.3 we show that performance remains robust when the rendering platform used for the attack differs from the one used to generate the target map.

Table 4: Location recovery error across map backgrounds.

Background	Method	OpenAddresses		Synthetic	
		Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
White canvas	PixelMatch	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	PixelAvg	236.11 $\pm$ 82.69	0.0874 $\pm$ 0.0306	226.44 $\pm$ 78.30	0.0839 $\pm$ 0.0290
	Raster2Vec	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	AutoLocate _{BU}	1.67 $\pm$ 0.61	0.0006 $\pm$ 0.0002	1.61 $\pm$ 0.50	0.0006 $\pm$ 0.0002
	AutoLocate _{BK}	0.91 $\pm$ 0.62	0.0003 $\pm$ 0.0002	0.81 $\pm$ 0.50	0.0002 $\pm$ 0.0001
Street	PixelMatch	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	PixelAvg	236.11 $\pm$ 82.69	0.0874 $\pm$ 0.0306	226.44 $\pm$ 78.30	0.0839 $\pm$ 0.0290
	Raster2Vec	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	AutoLocate _{BU}	1.81 $\pm$ 0.66	0.0007 $\pm$ 0.0003	1.69 $\pm$ 0.51	0.0006 $\pm$ 0.0002
	AutoLocate _{BK}	0.95 $\pm$ 0.64	0.0003 $\pm$ 0.0002	0.88 $\pm$ 0.52	0.0002 $\pm$ 0.0001
Satellite	PixelMatch	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	PixelAvg	236.11 $\pm$ 82.69	0.0874 $\pm$ 0.0306	226.44 $\pm$ 78.30	0.0839 $\pm$ 0.0290
	Raster2Vec	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	AutoLocate _{BU}	10.50 $\pm$ 3.64	0.0039 $\pm$ 0.0013	9.41 $\pm$ 3.25	0.0035 $\pm$ 0.0012
	AutoLocate _{BK}	2.13 $\pm$ 0.67	0.0008 $\pm$ 0.0002	2.73 $\pm$ 0.52	0.0010 $\pm$ 0.0002

Attack Setup. We use the Pillow library [17] to load target map images, process pixel data, and run our recovery algorithms. All evaluated map visualization platforms provide command-line interfaces, enabling automated map generation for our optimization procedure. We also leverage the coordinate-transformation functions provided by these platforms to convert between pixel and geographic coordinates. The recovery pipeline is fully automated and requires no human intervention or visual inspection.

5.2 Evaluation of AutoLocate (RQ1)

Performance Across Different Map Scales. We use a street map background and GeoPandas to generate target maps at three scales (*i.e.*, small, medium, and large) to evaluate location recovery performance. As shown in Table 3, all baseline methods (*i.e.*, PixelMatch, PixelAvg, and Raster2Vec) produce similar recovery errors on the order of hundreds of meters at the small scale, indicating that they are ineffective at recovering precise locations from dot maps covering broad geographic regions. In contrast, AutoLocate_{BK} and AutoLocate_{BU}, which exploit anti-aliasing artifacts, achieve errors of approximately 1 meter and 2 meters on the OpenAddresses dataset, respectively, representing more than 200 $\times$ and 100 $\times$ improvements over the strongest baseline. This demonstrates that precise locations can be recovered even from dot maps covering large geographic areas. We find that the recovery errors are higher on the OpenAddresses dataset, which we attribute to the complexity of real-world spatial distributions, where higher local dot densities make location estimation more challenging. This trend is consistent across scales and datasets, with our methods exhibiting similar relative pixel errors.

Performance Across Different Map Backgrounds. We vary the map background (*i.e.*, white canvas, street map, and satellite imagery) to examine its impact on recovery accuracy. As shown in Table 4, baseline methods yield consistently high recovery errors across all backgrounds, as they rely solely on dot color, failing to account for anti-aliasing effects arising from the blending of dots with the underlying background. In contrast, our proposed methods maintain high location recovery performance across all

Table 5: Location recovery error across map resolutions.

Resolution	Method	OpenAddresses		Synthetic	
		Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
384 DPI (4568 $\times$ 2848)	PixelMatch	65.12 $\pm$ 48.68	0.0482 $\pm$ 0.0360	64.23 $\pm$ 48.42	0.0476 $\pm$ 0.0359
	PixelAvg	53.11 $\pm$ 42.45	0.0393 $\pm$ 0.0314	52.56 $\pm$ 39.32	0.0389 $\pm$ 0.0291
	Raster2Vec	65.12 $\pm$ 48.68	0.0482 $\pm$ 0.0360	64.23 $\pm$ 48.42	0.0476 $\pm$ 0.0359
	AutoLocate _{BU}	0.98 $\pm$ 0.52	0.0007 $\pm$ 0.0004	0.66 $\pm$ 0.28	0.0005 $\pm$ 0.0002
	AutoLocate _{BK}	0.46 $\pm$ 0.33	0.0003 $\pm$ 0.0002	0.36 $\pm$ 0.31	0.0003 $\pm$ 0.0003
192 DPI (2284 $\times$ 1424)	PixelMatch	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	PixelAvg	236.11 $\pm$ 82.69	0.0874 $\pm$ 0.0306	226.44 $\pm$ 78.30	0.0839 $\pm$ 0.0290
	Raster2Vec	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	AutoLocate _{BU}	1.81 $\pm$ 0.66	0.0007 $\pm$ 0.0003	1.69 $\pm$ 0.51	0.0006 $\pm$ 0.0002
	AutoLocate _{BK}	0.95 $\pm$ 0.64	0.0003 $\pm$ 0.0002	0.88 $\pm$ 0.52	0.0002 $\pm$ 0.0001
96 DPI (1142 $\times$ 712)	PixelMatch	370.67 $\pm$ 224.32	0.0686 $\pm$ 0.0415	335.56 $\pm$ 186.80	0.0621 $\pm$ 0.0346
	PixelAvg	405.33 $\pm$ 192.50	0.0751 $\pm$ 0.0357	375.08 $\pm$ 144.41	0.0695 $\pm$ 0.0268
	Raster2Vec	370.67 $\pm$ 224.32	0.0686 $\pm$ 0.0415	335.56 $\pm$ 186.80	0.0621 $\pm$ 0.0346
	AutoLocate _{BU}	7.36 $\pm$ 3.07	0.0014 $\pm$ 0.0006	6.84 $\pm$ 2.04	0.0013 $\pm$ 0.0004
	AutoLocate _{BK}	4.28 $\pm$ 1.64	0.0008 $\pm$ 0.0003	3.84 $\pm$ 1.06	0.0007 $\pm$ 0.0002

Table 6: Location recovery error across different ground sample distances (GSD).

GSD (m/pixel)	Method	OpenAddresses		Synthetic	
		Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
27	AutoLocate _{BU}	0.05 $\pm$ 0.046	0.0019 $\pm$ 0.0002	0.05 $\pm$ 0.060	0.0019 $\pm$ 0.0006
	AutoLocate _{BK}	0.05 $\pm$ 0.052	0.0019 $\pm$ 0.0002	0.03 $\pm$ 0.080	0.0011 $\pm$ 0.0007
270	AutoLocate _{BU}	0.25 $\pm$ 0.06	0.0009 $\pm$ 0.0002	0.21 $\pm$ 0.06	0.0008 $\pm$ 0.0002
	AutoLocate _{BK}	0.12 $\pm$ 0.06	0.0004 $\pm$ 0.0002	0.10 $\pm$ 0.05	0.0004 $\pm$ 0.0002
1350	AutoLocate _{BU}	0.98 $\pm$ 0.52	0.0007 $\pm$ 0.0004	0.66 $\pm$ 0.28	0.0005 $\pm$ 0.0002
	AutoLocate _{BK}	0.46 $\pm$ 0.33	0.0003 $\pm$ 0.0002	0.36 $\pm$ 0.31	0.0003 $\pm$ 0.0003
2700	AutoLocate _{BU}	1.81 $\pm$ 0.66	0.0007 $\pm$ 0.0003	1.69 $\pm$ 0.51	0.0006 $\pm$ 0.0002
	AutoLocate _{BK}	0.95 $\pm$ 0.64	0.0003 $\pm$ 0.0002	0.88 $\pm$ 0.52	0.0002 $\pm$ 0.0001
5400	AutoLocate _{BU}	7.36 $\pm$ 3.07	0.0014 $\pm$ 0.0006	6.84 $\pm$ 2.04	0.0013 $\pm$ 0.0004
	AutoLocate _{BK}	4.28 $\pm$ 1.64	0.0008 $\pm$ 0.0003	3.84 $\pm$ 1.06	0.0007 $\pm$ 0.0002

settings. Both AutoLocate_{BK} and AutoLocate_{BU} exhibit a performance drop on highly complex satellite imagery. This degradation is more pronounced for AutoLocate_{BU}, as complex backgrounds reduce the accuracy of local background estimation. Nevertheless, AutoLocate_{BU} still significantly outperforms all baselines, even under these challenging conditions.

Performance Across Map Resolution. We further evaluate the impact of image resolution on recovery performance by rendering maps at three different resolutions: 384, 192, and 96 DPI. The results are shown in Table 5. As expected, the performance of all methods degrades at lower resolutions due to the reduced number of pixels available to estimate dot centroids accurately. Despite this, AutoLocate maintains strong performance even under low-resolution settings. At 96 DPI, AutoLocate_{BK} still achieves a recovery error of less than 5 meters, which remains sufficient for precise location recovery and continues to significantly outperform the baselines under the same conditions.

Joint Impact of Map Scale and Resolution. To study the impact of map scale and resolution jointly, we adopt a single metric, the ground sample distance (GSD), defined as the real-world distance (in meters) represented by one pixel. We vary the GSD from 27 to 5400 m/pixel by adjusting the map scale and resolution, and examine the recovery performance on both datasets, as shown in Table 6. As

Table 7: Comparison of the location recovery error (in meters) across different dot geometries and sizes.

Geometry	Method	OpenAddresses			Synthetic		
		1mm	2mm	3mm	1mm	2mm	3mm
Circle	PixelMatch	168.05 $\pm$ 149.42	287.08 $\pm$ 105.15	99.56 $\pm$ 68.73	181.08 $\pm$ 105.24	256.18 $\pm$ 97.74	126.84 $\pm$ 82.93
	PixelAvg	219.19 $\pm$ 134.29	236.11 $\pm$ 82.69	103.85 $\pm$ 55.04	196.25 $\pm$ 112.78	226.44 $\pm$ 78.30	102.16 $\pm$ 49.52
	Raster2Vec	168.05 $\pm$ 149.42	287.08 $\pm$ 105.15	99.56 $\pm$ 68.73	181.08 $\pm$ 105.24	256.18 $\pm$ 97.74	126.84 $\pm$ 82.93
	AutoLocate _{BU}	7.68 $\pm$ 4.98	1.81 $\pm$ 0.66	1.80 $\pm$ 1.15	5.33 $\pm$ 3.30	1.69 $\pm$ 0.51	1.60 $\pm$ 1.12
	AutoLocate _{BK}	1.73 $\pm$ 1.11	0.95 $\pm$ 0.64	0.94 $\pm$ 0.64	1.92 $\pm$ 1.16	0.88 $\pm$ 0.52	0.87 $\pm$ 0.57
Pentagon	PixelMatch	271.80 $\pm$ 163.90	426.26 $\pm$ 181.30	316.41 $\pm$ 190.00	300.77 $\pm$ 151.09	323.57 $\pm$ 171.63	306.25 $\pm$ 177.14
	PixelAvg	263.62 $\pm$ 162.24	392.14 $\pm$ 180.89	289.59 $\pm$ 176.50	293.30 $\pm$ 165.56	285.61 $\pm$ 174.83	265.63 $\pm$ 164.79
	Raster2Vec	271.80 $\pm$ 163.90	426.26 $\pm$ 181.30	316.41 $\pm$ 190.00	300.77 $\pm$ 151.09	323.57 $\pm$ 171.63	306.25 $\pm$ 177.14
	AutoLocate _{BU}	7.90 $\pm$ 5.63	6.61 $\pm$ 5.43	5.55 $\pm$ 2.88	10.49 $\pm$ 7.33	5.17 $\pm$ 2.71	4.47 $\pm$ 2.93
	AutoLocate _{BK}	4.04 $\pm$ 2.51	3.97 $\pm$ 2.56	2.25 $\pm$ 1.80	4.76 $\pm$ 2.89	3.70 $\pm$ 2.11	2.24 $\pm$ 1.57
Triangle	PixelMatch	441.29 $\pm$ 264.24	392.82 $\pm$ 200.83	513.14 $\pm$ 292.00	477.40 $\pm$ 281.77	406.01 $\pm$ 262.56	424.83 $\pm$ 254.01
	PixelAvg	466.58 $\pm$ 303.88	391.55 $\pm$ 192.16	463.76 $\pm$ 288.14	442.65 $\pm$ 284.13	387.77 $\pm$ 266.23	398.10 $\pm$ 267.11
	Raster2Vec	441.29 $\pm$ 264.24	392.82 $\pm$ 200.83	513.14 $\pm$ 292.00	477.40 $\pm$ 281.77	406.01 $\pm$ 262.56	424.83 $\pm$ 254.01
	AutoLocate _{BU}	15.18 $\pm$ 8.81	8.56 $\pm$ 5.04	7.68 $\pm$ 4.04	10.72 $\pm$ 6.41	7.43 $\pm$ 4.10	6.17 $\pm$ 3.42
	AutoLocate _{BK}	10.22 $\pm$ 5.30	6.78 $\pm$ 3.94	4.58 $\pm$ 2.39	8.21 $\pm$ 4.78	5.86 $\pm$ 3.48	4.09 $\pm$ 2.31

Table 8: Location recovery error across map tools.

Platform	Method	OpenAddresses		Synthetic	
		Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
GeoPandas	PixelMatch	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	PixelAvg	236.11 $\pm$ 82.69	0.0874 $\pm$ 0.0306	226.44 $\pm$ 78.30	0.0839 $\pm$ 0.0290
	Raster2Vec	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	AutoLocate _{BU}	1.81 $\pm$ 0.66	0.0007 $\pm$ 0.0003	1.69 $\pm$ 0.51	0.0006 $\pm$ 0.0002
	AutoLocate _{BK}	0.95 $\pm$ 0.64	0.0003 $\pm$ 0.0002	0.88 $\pm$ 0.52	0.0002 $\pm$ 0.0001
QGIS	PixelMatch	258.13 $\pm$ 99.80	0.0956 $\pm$ 0.0367	227.74 $\pm$ 82.40	0.0844 $\pm$ 0.0308
	PixelAvg	291.12 $\pm$ 96.40	0.1078 $\pm$ 0.0355	273.30 $\pm$ 98.70	0.1012 $\pm$ 0.0368
	Raster2Vec	258.13 $\pm$ 99.80	0.0956 $\pm$ 0.0367	227.74 $\pm$ 82.40	0.0844 $\pm$ 0.0308
	AutoLocate _{BU}	9.51 $\pm$ 3.21	0.0035 $\pm$ 0.0012	8.01 $\pm$ 2.70	0.0030 $\pm$ 0.0010
	AutoLocate _{BK}	9.14 $\pm$ 5.54	0.0034 $\pm$ 0.0020	2.86 $\pm$ 1.91	0.0011 $\pm$ 0.0007
R	PixelMatch	262.53 $\pm$ 91.70	0.0972 $\pm$ 0.0342	261.67 $\pm$ 105.60	0.0969 $\pm$ 0.0390
	PixelAvg	291.76 $\pm$ 109.30	0.1080 $\pm$ 0.0406	220.71 $\pm$ 72.50	0.0817 $\pm$ 0.0269
	Raster2Vec	262.53 $\pm$ 91.70	0.0972 $\pm$ 0.0342	261.67 $\pm$ 105.60	0.0969 $\pm$ 0.0390
	AutoLocate _{BU}	4.01 $\pm$ 1.58	0.0015 $\pm$ 0.0006	3.93 $\pm$ 1.08	0.0015 $\pm$ 0.0004
	AutoLocate _{BK}	2.58 $\pm$ 1.61	0.0010 $\pm$ 0.0006	2.50 $\pm$ 1.57	0.0009 $\pm$ 0.0006

expected, the distance error of AutoLocate grows approximately in proportion to the GSD, since each pixel covers a larger geographic area. In contrast, the relative pixel error of both methods remains at the millipixel level across all GSDs, indicating that AutoLocate recovers dot centroids with a stable sub-pixel precision.

Performance Across Dot Properties. We vary dot geometry and size to examine their impact on recovery accuracy. As shown in Table 7, the performance of all methods degrades as dot shapes become more complex (*e.g.*, from circles to pentagons). We observe two distinct trends with respect to dot size. First, as the dot size increases, the accuracy of our methods improves, while the performance of the baselines typically deteriorates. This is because larger dots introduce more anti-aliased boundary pixels, providing richer sub-pixel information that our optimization can exploit, whereas baseline methods fail to benefit from this additional signal. Second, AutoLocate remains effective even for very small dots (*i.e.*, 1 millimeter in size, corresponding to only about 4 pixels), significantly outperforming the baselines in this challenging regime.

Performance Across Map Visualization Platforms. We evaluate our attack on dot maps generated by three widely used visualization platforms: GeoPandas, QGIS, and R. As shown in Table 8, our methods achieve strong recovery performance, with accuracy within 10 meters. This indicates that, despite potential differences

Table 9: Location recovery error across map formats.

Map Format	Method	OpenAddresses		Synthetic	
		Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
PNG	PixelMatch	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	PixelAvg	236.11 $\pm$ 82.69	0.0874 $\pm$ 0.0306	226.44 $\pm$ 78.30	0.0839 $\pm$ 0.0290
	Raster2Vec	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	AutoLocate _{BU}	1.81 $\pm$ 0.66	0.0007 $\pm$ 0.0003	1.69 $\pm$ 0.51	0.0006 $\pm$ 0.0002
	AutoLocate _{BK}	0.95 $\pm$ 0.64	0.0003 $\pm$ 0.0002	0.88 $\pm$ 0.52	0.0002 $\pm$ 0.0001
TIFF	PixelMatch	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	PixelAvg	236.11 $\pm$ 82.69	0.0874 $\pm$ 0.0306	226.44 $\pm$ 78.30	0.0839 $\pm$ 0.0290
	Raster2Vec	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	AutoLocate _{BU}	1.81 $\pm$ 0.66	0.0007 $\pm$ 0.0003	1.69 $\pm$ 0.51	0.0006 $\pm$ 0.0002
	AutoLocate _{BK}	0.95 $\pm$ 0.64	0.0003 $\pm$ 0.0002	0.88 $\pm$ 0.52	0.0002 $\pm$ 0.0001
JPEG	PixelMatch	320.53 $\pm$ 88.07	0.1187 $\pm$ 0.0326	333.03 $\pm$ 91.79	0.1233 $\pm$ 0.0340
	PixelAvg	308.76 $\pm$ 60.77	0.1144 $\pm$ 0.0225	294.37 $\pm$ 66.22	0.1090 $\pm$ 0.0245
	Raster2Vec	320.53 $\pm$ 88.07	0.1187 $\pm$ 0.0326	333.03 $\pm$ 91.79	0.1233 $\pm$ 0.0340
	AutoLocate _{BU}	7.46 $\pm$ 26.87	0.0028 $\pm$ 0.0072	7.12 $\pm$ 6.68	0.0026 $\pm$ 0.0024
	AutoLocate _{BK}	6.27 $\pm$ 5.09	0.0023 $\pm$ 0.0018	5.84 $\pm$ 5.17	0.0022 $\pm$ 0.0019

Table 10: Location recovery error of isolated dots and overlapping dots.

# Overlaps	Method	OpenAddresses		Synthetic	
		Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
1	PixelMatch	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	PixelAvg	236.11 $\pm$ 82.69	0.0874 $\pm$ 0.0306	226.44 $\pm$ 78.30	0.0839 $\pm$ 0.0290
	Raster2Vec	287.08 $\pm$ 105.15	0.1063 $\pm$ 0.0389	256.18 $\pm$ 97.74	0.0949 $\pm$ 0.0362
	AutoLocate _{BU}	1.81 $\pm$ 0.66	0.0007 $\pm$ 0.0003	1.69 $\pm$ 0.51	0.0006 $\pm$ 0.0002
2	AutoLocate _{BK}	0.95 $\pm$ 0.64	0.0003 $\pm$ 0.0002	0.88 $\pm$ 0.52	0.0002 $\pm$ 0.0001
	AutoLocate _{BU}	7.34 $\pm$ 102.29	0.0027 $\pm$ 0.0376	3.00 $\pm$ 1.66	0.0011 $\pm$ 0.0005
	AutoLocate _{BK}	2.26 $\pm$ 1.04	0.0008 $\pm$ 0.0004	1.82 $\pm$ 0.70	0.0007 $\pm$ 0.0003
	AutoLocate _{BU}	8.89 $\pm$ 185.73	0.0033 $\pm$ 0.0689	4.79 $\pm$ 50.39	0.0018 $\pm$ 0.0189
3	AutoLocate _{BK}	4.31 $\pm$ 32.54	0.0016 $\pm$ 0.0121	2.17 $\pm$ 0.99	0.0008 $\pm$ 0.0004
	AutoLocate _{BU}	27.00 $\pm$ 1046.78	0.0100 $\pm$ 0.3877	4.39 $\pm$ 161.92	0.0016 $\pm$ 0.0590
	AutoLocate _{BK}	4.86 $\pm$ 241.87	0.0018 $\pm$ 0.0896	2.50 $\pm$ 15.54	0.0009 $\pm$ 0.0056
	AutoLocate _{BU}	16.12 $\pm$ 11828.85	0.0060 $\pm$ 4.4028	5.37 $\pm$ 641.73	0.0020 $\pm$ 0.2390
5	AutoLocate _{BK}	5.46 $\pm$ 10264.44	0.0020 $\pm$ 3.7599	2.92 $\pm$ 1158.51	0.0011 $\pm$ 0.4364

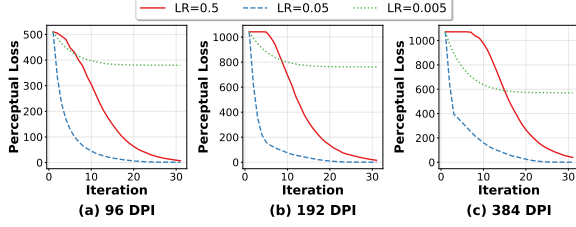
in underlying (and often unknown) rendering processes, our attack remains robust and achieves high-precision location recovery regardless of the visualization software used.

Performance Across Image Formats. Dot maps are exported in different image formats for dissemination. We evaluate our attack on maps saved in three common formats: PNG, TIFF, and JPEG. As shown in Table 9, recovery performance is identical for PNG and TIFF, which is expected since both preserve RGB values without compression artifacts. In contrast, all methods exhibit degraded performance on JPEG images, likely due to compression losses that distort pixel-level information. Despite this, our methods still outperform all baselines by nearly 50 $\times$ under JPEG compression. Overall, these results demonstrate that our methods generalize well across common raster formats and maintain high-precision location recovery, regardless of the map format.

Performance on Overlapping Dots. As shown in Table 10, median recovery error generally increases with overlap, since merged regions blur boundaries and make it harder to leverage anti-aliasing artifacts for accurate centroid estimation. AutoLocate_{BK} degrades only slightly in terms of median error, maintaining a median recovery error below 6 meters even with five overlapping dots across both datasets. In contrast, AutoLocate_{BU} shows a larger degradation, with median recovery error increasing from approximately

Table 11: Location recovery performance in Mexico.

Method	OpenAddresses		Synthetic	
	Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
PixelMatch	157.34 $\pm$ 51.00	0.1075 $\pm$ 0.0348	162.74 $\pm$ 55.54	0.1112 $\pm$ 0.0379
PixelAvg	132.43 $\pm$ 39.52	0.0905 $\pm$ 0.0270	141.70 $\pm$ 46.62	0.0968 $\pm$ 0.0319
Raster2Vec	157.34 $\pm$ 51.00	0.1075 $\pm$ 0.0348	162.74 $\pm$ 55.54	0.1112 $\pm$ 0.0379
AutoLocate _{BU}	0.69 $\pm$ 0.33	0.0005 $\pm$ 0.0002	0.76 $\pm$ 0.34	0.0005 $\pm$ 0.0002
AutoLocate _{BK}	0.60 $\pm$ 0.35	0.0004 $\pm$ 0.0002	0.73 $\pm$ 0.31	0.0005 $\pm$ 0.0002

**Figure 5: Impact of iterations and step size in AutoLocate_{BK}.**

2 meters for isolated dots to as high as 27 meters under overlap. Nevertheless, even with five overlapping dots, the median recovery errors of our methods remain substantially lower than those of the baselines on the much simpler task of recovering isolated dots.

Performance Across Geographic Regions. We further evaluate our attack on maps of Mexico to test whether it generalizes beyond the U.S. Specifically, we sample coordinates from OpenAddresses and generate synthetic data using the same procedure as for the U.S. datasets, and render the maps with all other configurations set to their defaults. As shown in Table 11, AutoLocate_{BK} recovers locations to within 0.60 m and 0.73 m on OpenAddresses and Synthetic, and AutoLocate_{BU} is likewise sub-meter. In contrast, all baselines produce errors on the order of hundreds of meters. These results show that our attack generalizes across geographic regions.

5.3 Ablation Study (RQ2)

Impact of Optimization Hyperparameters. We analyze the effect of the optimization iterations T and step size η on the convergence and recovery performance of AutoLocate_{BK} (results for AutoLocate_{BU} are omitted due to similar trends). Figure 5 shows the perceptual loss trajectories at different resolutions under varying step sizes. We find that a large step size (*i.e.*, 0.5) fails to converge and yields inaccurate estimates, while a small step size (*i.e.*, 0.005) converges too slowly. Overall, our default setting (*i.e.*, $\eta = 0.05$) converges reliably within 30 iterations and achieves the lowest perceptual loss. These results are consistent across map resolutions, highlighting the stability of our algorithm.

Impact of Background in AutoLocate_{BK}. In AutoLocate_{BK}, perceptual loss is computed by rendering a new map using the available background and the currently estimated dot centroids. In practice, the adversary may not have access to the exact background used in the target map. We therefore evaluate how background variations affect recovery performance. Specifically, we use a street map from OpenStreetMap as the ground-truth background and consider three

Table 12: Impact of different map backgrounds on the location recovery error of AutoLocate_{BK}.

Background	OpenAddresses		Synthetic	
	Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
Same	0.95 $\pm$ 0.64	0.0003 $\pm$ 0.0002	0.88 $\pm$ 0.52	0.0002 $\pm$ 0.0001
Similar	8.92 $\pm$ 5.75	0.0033 $\pm$ 0.0021	8.62 $\pm$ 5.39	0.0032 $\pm$ 0.0020
Perturbed ($\sigma = 5$)	11.37 $\pm$ 7.39	0.0042 $\pm$ 0.0027	10.93 $\pm$ 6.89	0.0040 $\pm$ 0.0026
Perturbed ($\sigma = 10$)	19.48 $\pm$ 12.37	0.0072 $\pm$ 0.0046	18.64 $\pm$ 12.21	0.0069 $\pm$ 0.0045
Perturbed ($\sigma = 15$)	29.86 $\pm$ 19.71	0.0111 $\pm$ 0.0072	27.04 $\pm$ 17.31	0.0100 $\pm$ 0.0065
Perturbed ($\sigma = 20$)	28.87 $\pm$ 18.62	0.0107 $\pm$ 0.0070	31.35 $\pm$ 19.59	0.0116 $\pm$ 0.0071
Perturbed ($\sigma = 25$)	35.04 $\pm$ 22.78	0.0130 $\pm$ 0.0083	35.29 $\pm$ 22.59	0.0131 $\pm$ 0.0083

Table 13: Impact of rendering platform for AutoLocate across datasets. The target map is generated using GeoPandas.

Platform	Method	OpenAddresses		Synthetic	
		Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
GeoPandas	AutoLocate _{BU}	1.81 $\pm$ 0.66	0.0007 $\pm$ 0.0003	1.69 $\pm$ 0.51	0.0006 $\pm$ 0.0002
	AutoLocate _{BK}	0.95 $\pm$ 0.64	0.0003 $\pm$ 0.0002	0.88 $\pm$ 0.52	0.0002 $\pm$ 0.0001
QGIS	AutoLocate _{BU}	2.67 $\pm$ 1.21	0.0010 $\pm$ 0.0006	2.36 $\pm$ 1.70	0.0009 $\pm$ 0.0007
	AutoLocate _{BK}	2.35 $\pm$ 1.54	0.0009 $\pm$ 0.0006	2.41 $\pm$ 1.91	0.0009 $\pm$ 0.0007
R	AutoLocate _{BU}	2.51 $\pm$ 1.58	0.0009 $\pm$ 0.0006	2.49 $\pm$ 1.08	0.0009 $\pm$ 0.0004
	AutoLocate _{BK}	2.28 $\pm$ 1.61	0.0008 $\pm$ 0.0006	2.18 $\pm$ 1.57	0.0008 $\pm$ 0.0006

scenarios: (i) *Same*: the background is identical to that of the target map; (ii) *Similar*: a different street map of the same region from Esri [31], captured in a different year; (iii) *Perturbed*: the background is corrupted with additive Gaussian noise (*i.e.*, $\mathcal{N}(0, \sigma^2)$) applied independently to each RGB channel (with σ varying from 5 to 25), while preserving overall visual appearance.

The results are shown in Table 12. We find that even small deviations from the true background (*e.g.*, perturbations with $\sigma = 5$) lead to noticeable drops in recovery accuracy. This arises because AutoLocate_{BK} depends on subtle anti-aliasing artifacts along dot boundaries; any discrepancy in the background disrupts these cues, resulting in inaccurate loss evaluation and suboptimal optimization. When the background differs substantially from the ground truth (*e.g.*, large perturbations or different map sources), the performance of AutoLocate_{BK} falls below that of AutoLocate_{BU}, which directly estimates the background from the target map. In such cases, we recommend using AutoLocate_{BU} when the background is unknown or cannot be accurately reproduced.

Impact of Map Visualization Tools Used in AutoLocate. In our previous experiments, we assumed that the adversary has access to the same visualization tool (*i.e.*, rendering function $\mathcal{R}$) used to generate the target map. In practice, however, this assumption may not hold. To evaluate this scenario, we fix the target map to one generated using GeoPandas with a street map background and vary the rendering tool used by the adversary during optimization. The results in Table 13 show that mismatches between the target and adversary tools have only a minor impact on recovery accuracy, with location recovery errors remaining within 3 meters for both AutoLocate_{BK} and AutoLocate_{BU}. This demonstrates that AutoLocate is robust even when the adversary does not know, or have access to, the specific tool used to generate the target map.

Efficiency Evaluation. We evaluate the efficiency of our recovery algorithms and compare them with the baselines on a laptop equipped with a Snapdragon X Elite CPU. As shown in Table 14,

Table 14: Running time (in minutes) comparison of attacks.

Map Resolution	PixelMatch	PixelAvg	Raster2Vec	AutoLocate _{BU}	AutoLocate _{BK}
1142 × 712	0.10	0.09	0.10	1.57	1.30
2284 × 1424	0.18	0.18	0.18	3.58	3.76
4568 × 2848	0.45	0.47	0.45	8.21	8.12

Table 15: Location recovery error of AutoLocate at different levels of input coordinate precision.

Coordinate Precision	Method	OpenAddresses		Synthetic	
		Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
4 decimals	AutoLocate _{BU}	5.74 ± 1.92	0.0021 ± 0.0007	5.67 ± 1.41	0.0021 ± 0.0005
	AutoLocate _{BK}	3.52 ± 1.68	0.0013 ± 0.0006	3.46 ± 1.42	0.0013 ± 0.0005
5 decimals	AutoLocate _{BU}	2.00 ± 0.67	0.0007 ± 0.0002	1.87 ± 0.54	0.0007 ± 0.0002
	AutoLocate _{BK}	1.32 ± 0.66	0.0005 ± 0.0003	1.24 ± 0.55	0.0005 ± 0.0002
6 decimals	AutoLocate _{BU}	1.81 ± 0.68	0.0007 ± 0.0003	1.69 ± 0.51	0.0006 ± 0.0002
	AutoLocate _{BK}	0.95 ± 0.64	0.0003 ± 0.0002	0.88 ± 0.52	0.0002 ± 0.0001

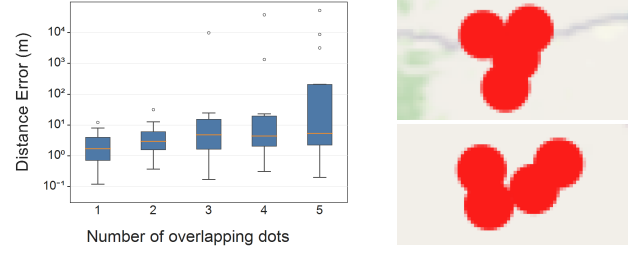
Table 16: Location recovery error (meters) of AutoLocate using the four-neighbor boundary set and the extended set that further includes the four-neighbors of the boundary pixels.

Background	Method	OpenAddresses		Synthetic	
		Four-neighbor	Extended	Four-neighbor	Extended
White canvas	AutoLocate _{BU}	1.67 ± 0.61	1.72 ± 0.59	1.61 ± 0.50	1.68 ± 0.50
	AutoLocate _{BK}	0.91 ± 0.62	0.91 ± 0.59	0.81 ± 0.50	0.81 ± 0.49
Street	AutoLocate _{BU}	1.81 ± 0.66	1.80 ± 0.64	1.69 ± 0.51	1.65 ± 0.53
	AutoLocate _{BK}	0.95 ± 0.64	0.95 ± 0.64	0.88 ± 0.52	0.88 ± 0.54
Satellite	AutoLocate _{BU}	10.50 ± 3.64	10.41 ± 3.61	9.41 ± 3.25	9.41 ± 3.11
	AutoLocate _{BK}	2.13 ± 0.67	2.13 ± 0.67	2.73 ± 0.52	2.73 ± 0.52

the baseline methods are highly efficient, requiring less than one minute to estimate dot centroids across different resolutions. Our methods (AutoLocate_{BK} and AutoLocate_{BU}) incur a higher computational cost, taking several minutes to complete. We find that this overhead is dominated by repeated map rendering during optimization, particularly at higher resolutions. Nevertheless, the overall runtime remains practical, requiring only a few minutes on a laptop. These results demonstrate that our attacks are feasible for adversaries without access to specialized computational resources.

Impact of Input Coordinate Precision. To evaluate whether reduced input coordinate precision affects attack performance, we vary the decimal precision of the input coordinates from 4 to 6 digits when generating the dot maps (where 4 decimal places correspond to approximately 11 meters of spatial precision). For each precision level, we generate target maps and measure recovery error with respect to the original high-precision coordinates. As shown in Table 15, reducing precision to 4 or 5 decimal places has only a modest impact on recovery accuracy, with errors remaining below 6 meters across both datasets. This still significantly outperforms all baselines, which incur errors on the order of hundreds of meters. These results demonstrate that AutoLocate remains effective even when the input coordinates are provided at coarse precision.

Impact of Boundary Pixel Selection. By default, AutoLocate computes the perceptual loss over the boundary pixels S , *i.e.*, the four-neighbors of the inner pixels. To evaluate whether AutoLocate



(a) Box plot illustrating the distribution of recovery distance error across varying numbers of overlapping dots. Outliers are represented as circles. (b) Top: Hard-to-attack overlapping dots. Bottom: Easy-to-attack overlapping dots.

Figure 6: (a) Analysis of recovery error distribution across varying numbers of overlapping dots. (b) Examples of easy-to-attack and hard-to-attack overlapping dots.

can benefit from pixels farther from the dot, we extend S by including the four-neighbors of the boundary pixels themselves (excluding inner pixels), and compare the resulting recovery performance. As shown in Table 16, we observe no noticeable performance difference, suggesting that the default four-neighbor boundary set is sufficient for accurate location recovery with AutoLocate.

5.4 In-depth Analysis (RQ3)

While the previous experiments mainly examined privacy risks in dot maps by considering recovery error at an aggregate level, privacy can also be viewed through a worst-case lens [25, 56]. In this section, we take a finer-grained view by analyzing the variability of recovery errors across individual dots, investigating why some dots are more vulnerable to attacks while others remain resistant.

Recovery Error Distribution. We first plot the distribution of recovery errors for all dots on a small-scale map using AutoLocate_{BU}. As shown in Figure 6(a), we observe that dots exhibit different recovery difficulties. While the median error is under 2 meters, some “easy-to-attack” instances (in the first quartile of the box plot) can be recovered with near-perfect accuracy. Conversely, “hard-to-attack” instances (above the third quartile) exhibit greater resilience against our attacks. Notably, overlapping dots, especially those with five overlaps, exhibit higher recovery errors compared to isolated dots. This suggests two key points: (i) the privacy risks associated with individual dots are not uniform across the map, and (ii) a large portion of dots in the map are vulnerable to our attacks because their recovery error is within a small range. In the following analysis, we conduct case studies on both easy-to-attack and hard-to-attack dots to explore the underlying reasons for this disparity.

Why Some Dots are Hard to Attack? We select a representative hard-to-attack dot with high recovery error, shown in the top part of Figure 6(b). This dot lies in a tightly clustered region where individual anti-aliasing boundary pixels are less distinct. Because the perceptual loss relies on boundary pixels, having only a few valid ones makes it difficult to accurately estimate the visual mismatch between the generated and target maps. This effect is most severe for dots at the center of a cluster, where nearly all boundary pixels are shared with neighboring dots, leaving little information for

Table 17: Location recovery error (meters) of AutoLocate under varying contrast and complexity of nearby background.

Complexity	Method	OpenAddresses		Synthetic	
		High Contrast	Low Contrast	High Contrast	Low Contrast
High	AutoLocate _{BU}	1.14 $\pm$.71	1.97 $\pm$.90	1.66 $\pm$.79	1.73 $\pm$.70
	AutoLocate _{BK}	0.68 $\pm$.31	0.79 $\pm$.47	0.95 $\pm$.48	0.89 $\pm$.35
Low	AutoLocate _{BU}	2.00 $\pm$.76	1.59 $\pm$.70	1.65 $\pm$.77	1.72 $\pm$.73
	AutoLocate _{BK}	1.20 $\pm$.54	0.98 $\pm$.40	0.83 $\pm$.43	0.94 $\pm$.27

precise localization. As a result, recovery errors for such dots are significantly higher.

Why Some Dots are Easy to Attack? We select a representative example from the lower quartile of the recovery error distribution, focusing on dots with four overlaps. As shown in the bottom part of Figure 6(b), although they also form a cluster of four dots, these dots are more widely separated, preserving a more complete ring of anti-aliasing boundary pixels around each centroid. This provides a stronger signal for the perceptual loss, enabling more accurate measurement of the visual mismatch between the generated and target maps. Furthermore, the background beneath these dots is visually simple, with sharp contrast between the dot color and the background, making the anti-aliasing artifacts particularly prominent. Together, these factors allow the adversary to estimate dot centroids with extremely high precision.

Impact of Background Complexity and Contrast. We further study how the local background affects attack performance. Specifically, we describe each isolated dot by two properties of the background at its boundary pixels: (i) *complexity*, the standard deviation of the background colors, and (ii) *contrast*, the average color difference between these background pixels and the dot color. We bin each property into low and high using the median as the threshold across both datasets (*i.e.*, 9.13 for complexity and 479.25 for contrast) and evaluate our methods on the street-map background. As shown in Table 17, both attacks remain accurate and robust across all four categories.

6 Mitigation Strategies

Our findings suggest that even seemingly benign dot maps that cover a large geographic region can reveal concealed, high-precision location information through our attacks. Notably, in many maps, we were able to pinpoint a significant proportion of individuals' locations with accuracy within 1 meter. Thus, rigorous guidelines are needed to ensure the safe publication of dot maps. In this section, we discuss several potential mitigation strategies and propose a risk assessment tool to evaluate the privacy risks of dot maps.

Potential Mitigation Strategies. We consider the following mitigation strategies to defend against location recovery attacks:

- *Publishing Maps Without Anti-Aliasing.* While anti-aliasing is enabled by default in all map visualization tools we are aware of, some tools (*e.g.*, QGIS) allow it to be disabled. Since AutoLocate relies on anti-aliasing artifacts to reverse-engineer dot locations, disabling it could be an effective defense against our attacks.
- *Geo-masking.* A significant body of work [13, 52, 80] has developed geo-masking techniques to mitigate privacy risks in location

Table 18: Mitigation strategies against AutoLocate_{BK}.

Mitigation	OpenAddresses		Synthetic	
	Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
w/o Anti-aliasing	235.98 $\pm$ 166.52	0.0874 $\pm$.0656	232.31 $\pm$ 105.65	0.0860 $\pm$.0416
Geo-masking (radius: 100 m)	100.33 $\pm$.91	0.0372 $\pm$.0004	100.21 $\pm$.93	0.0371 $\pm$.0004
Geo-masking (radius: 50 m)	49.92 $\pm$.85	0.0185 $\pm$.0003	50.16 $\pm$.87	0.0186 $\pm$.0003
Quantization (3 Decimals)	36.32 $\pm$ 12.18	0.0135 $\pm$.0048	35.26 $\pm$ 12.75	0.0131 $\pm$.0050
Quantization (2 Decimals)	400.75 $\pm$ 141.60	0.1484 $\pm$.0558	393.76 $\pm$ 123.64	0.1458 $\pm$.0487
None	0.95 $\pm$.64	0.0003 $\pm$.0002	0.88 $\pm$.52	0.0002 $\pm$.0001

Table 19: Effectiveness of the proposed assessment tool.

Anonymity Level k	Recommended Decimals	Anonymity Set Size
10	2.44 $\pm$.70	168.25 $\pm$ 268.82
20	2.15 $\pm$.63	556.96 $\pm$ 649.06
50	1.90 $\pm$.51	1,204.71 $\pm$ 2,129.87
100	1.85 $\pm$.47	1,472.24 $\pm$ 2,280.86

data. Here, we employ a simple approach [48, 90] that adds random noise to the raw location data by displacing each point in a random direction within a fixed radius (50 or 100 meters).

- *Location Quantization.* Another straightforward mitigation involves reducing the precision of the original location data (*e.g.*, from seven decimal places to three or two), shifting dots from their true positions, and thereby limiting recovery accuracy.

Note that these mitigation strategies are chosen because they are simple and widely known. While more sophisticated methods exist [9, 74, 90], we focus on these approaches to illustrate the effectiveness of representative mitigation classes against our attacks.

Mitigation Performance. We apply these mitigations to a small-scale map and evaluate the performance of our attacks. The results are shown in Table 18 and Table 22, respectively. All three strategies effectively degrade recovery accuracy, increasing errors from around 1 meter to tens or hundreds of meters and rendering the attack ineffective for identifying individual locations.

Privacy Risk Assessment Tool. While privacy regulations [3, 32, 79] mandate the protection of sensitive data, they lack concrete guidelines for dot map publishing. Motivated by this, we propose a privacy risk assessment tool to help researchers evaluate the privacy risks of their dot maps. We note that the success of location recovery depends on two factors: (1) the recovery accuracy of the attack and (2) the population density of the geographic region.

Based on this, we develop a privacy assessment tool that adaptively adjusts location quantization according to local population density. The tool takes as input a set of GPS locations, a user-specified anonymity level k , and a publicly available population density layer; in our implementation, we use WorldPop [76]. For each location, it identifies the finest coordinate precision (*i.e.*, the largest number of decimal places) such that the corresponding spatial cell is expected to contain at least k residents. The released location is indistinguishable from those of at least k residents, thereby achieving k -anonymity [81]. This approach offers practitioners an intuitive and flexible way to balance privacy protection with map usability while supporting compliance with privacy regulations such as the GDPR [32], CCPA [79], and HIPAA [3].

Effectiveness of the Assessment Tool. We evaluate whether the assessment tool can mitigate our attack. Specifically, for each anonymity level k , we run the tool on the OpenAddresses dataset to obtain the recommended coordinate precision of every location, *i.e.*, the number of decimal places kept in its GPS latitude and longitude. We then quantize each location to its recommended precision, render the resulting map, and attack it with AutoLocate_{BK}. For each recovered location, we measure its anonymity set size, *i.e.*, the number of residents within the spatial cell containing the recovered coordinate. Both metrics are averaged over all dots on the map. As shown in Table 19, increasing k leads the tool to retain fewer decimal places, from 2.44 at $k = 10$ to 1.85 at $k = 100$. Because the tool assigns an integer precision to each location, three and two decimal places correspond to approximately hundred-meter- and kilometer-scale spatial precision, respectively, while one decimal place corresponds to approximately ten-kilometer-scale precision. Thus, this quantization could potentially reduce utility for large-scale maps, while having less impact on maps intended only to show broad regional trends. Meanwhile, the mean anonymity set size increases from 168.25 to 1,472.24 residents as k increases from 10 to 100. The mean anonymity set size exceeds the target k in all settings, with the larger margins arising because many dots fall in densely populated areas. These results demonstrate that the tool effectively mitigates our attack while making the resulting privacy–utility trade-off explicit.

7 Related Work

Dot maps are increasingly popular tools for visualizing the spatial distribution of individuals and events [61, 75, 77]. In articles and publications, dot maps are most commonly shared as raster images, reflecting the conventions of print media and the convenience of distributing fixed image formats [26, 43, 62].

Privacy Risks with Dot Maps. Dot maps are frequently used to display sensitive personal data, such as patient locations and crime incident locations. For example, Armstrong [5] highlighted that in epidemiological and criminal investigations, it is common for dot maps to have a one-to-one correspondence between each dot and a specific case. A significant body of research [11, 22, 45, 54, 90] demonstrates that these dots can be reverse-engineered to re-identify precise locations, posing serious privacy risks. For instance, Brownstein et al. [11] found that over 26% of locations from presentation-quality maps and over 79% from publication maps could be accurately identified. Kounadi et al. [45] identified 41 articles between 2005 and 2012 that disclosed over 68,000 home addresses. These studies raise ethical and security concerns, especially for individuals with stigmatized conditions (*e.g.*, mental illness), as they could be targeted.

Most existing work focuses on large-scale maps that cover a limited geographic area, with little research addressing the privacy risks of small-scale dot maps that span broader regions. Such maps have been created for regions such as Germany and the Netherlands [77], Cameroon [84], and Thailand [63], as listed in [75]. While dot maps covering larger regions are not uncommon, their privacy implications remain largely unexplored. Furthermore, existing geo-location privacy studies have not explored the use of anti-aliasing for location recovery. This is a key focus of our work, where we

investigate how these map rendering techniques can be leveraged to recover high-precision location information.

Privacy Protection Strategies for Locations and Maps. Many studies propose geo-masking strategies to mitigate privacy risks in location-based data. One early approach is dot aggregation, where dot locations are aggregated at either the midpoint of the street segment or at the nearest street intersection [45, 52]. Another common technique is random perturbation, which introduces random noise to location coordinates. Various perturbation methods have been studied, including random direction and fixed radius [48, 90], random perturbation within a circle [6, 91], Gaussian displacement [14, 91], donut masking [60, 80], and bimodal Gaussian displacement [13]. Several studies extend quantitative privacy notions, such as k -anonymity [81] and differential privacy [25], to geo-location data, and develop location-preserving techniques [9, 28, 74, 86].

Deploying these defenses for dot map publications requires understanding the trade-off between privacy and map usability. While broad privacy regulations [3, 32, 79] mandate the protection of personal locations, they lack technical specifications for visual data dissemination. One needs to choose an appropriate defense level to satisfy privacy requirements while achieving good visualization readability. To address this, we introduce a risk assessment framework. Using a population density map, researchers can select an appropriate coordinate quantization precision tailored to their specific privacy and utility requirements, offering a flexible trade-off between privacy and map usability.

Image Vectorization and Deblurring. The computer graphics community has studied the problem of recovering vector representations from raster images, commonly referred to as image vectorization or deblurring [24, 39, 44, 83, 88]. These methods aim to reconstruct smooth, resolution-independent geometric shapes from pixelated inputs. While not designed for privacy analysis, these works share our observation that anti-aliasing encodes sub-pixel information. However, these approaches differ from ours in several fundamental ways. First, their objective is visual reconstruction (*e.g.*, recovering region topology and color palettes), whereas ours is the extraction of high-precision geographic coordinates. Second, they treat anti-aliasing as visual degradation to be eliminated in pursuit of sharp boundaries, whereas we exploit it to reverse-engineer the precise location. Third, advanced vectorization approaches [57] often require a white-box differentiable rendering pipeline, whereas our framework treats the map renderer as a black box, making it applicable to any visualization tool. Our empirical evaluation demonstrates that applying a standard vectorization tool (*i.e.*, Raster2Vec in QGIS) to our task yields location-recovery errors orders of magnitude larger than those of our proposed methods.

8 Conclusion

In this paper, we systematically study the privacy risks of dot maps by proposing AutoLocate, an automated high-precision location recovery framework. AutoLocate is an optimization-based algorithm that exploits anti-aliasing artifacts in dot maps for precise location estimation. Extensive experiments across different datasets and map configurations demonstrate the effectiveness and robustness

of the proposed method. We also explore several mitigation strategies and introduce a privacy assessment tool to help practitioners evaluate and mitigate the privacy risks of their dot maps. Our work reveals a new attack vector for recovering highly precise location information from dot maps and opens new directions for analyzing the privacy risks of spatial data visualizations.

Acknowledgments

This work was funded in part by the National Science Foundation (NSF) awards CNS-2207204 and CNS-2247794. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the sponsors.

References

- [1] 2025. OpenAddresses: The Free and Open Global Address Collection. <https://openaddresses.io/>.
- [2] 2025. pdfimages: PDF Image Extractor. <https://poppler.freedesktop.org/>.
- [3] 104th United States Congress. 1996. Health Insurance Portability and Accountability Act of 1996. *Public Law* 104 (1996), 191.
- [4] Francesco Agostinelli, Margaux Lufade, and Paolo Martellini. 2024. *On the spatial determinants of educational access*. Technical Report. National Bureau of Economic Research.
- [5] Marc Armstrong. 2002. Geographic information technologies and their potentially erosive effects on personal privacy. *Studies in the Social Sciences* 27 (01 2002).
- [6] Marc P Armstrong, Gerard Rushton, and Dale L Zimmerman. 1999. Geographically masking health data to preserve confidentiality. *Statistics in medicine* 18, 5 (1999), 497–525.
- [7] Richard A. Becker, Allan R. Wilks, and Ray Brownrigg. 2023. maps: Draw Geographical Maps. <https://cran.r-project.org/web/packages/maps/index.html>.
- [8] Giacomo Bilotti, Michael Kempf, Eljas Oksanen, Lizzie Scholtus, and Oliver Nakoinz. 2024. Point Pattern Analysis (PPA) as a tool for reproducible archaeological site distribution analyses and location processes in early iron age south-west Germany. *PLoS One* 19, 3 (2024).
- [9] Vincent Bindschadler and Reza Shokri. 2016. Synthesizing plausible privacy-preserving location traces. In *2016 IEEE symposium on security and privacy (SP)*. IEEE, 546–563.
- [10] John S. Brownstein, Christopher A. Cassa, Isaac S. Kohane, and Kenneth D. Mandl. 2005. Reverse geocoding: concerns about patient confidentiality in the display of geospatial health data. *AMIA Annual Symposium Proceedings* 2005 (2005), 905.
- [11] John S. Brownstein, Christopher A. Cassa, Isaac S. Kohane, and Kenneth D. Mandl. 2006. An Unsupervised Classification Method for Inferring Original Case Locations from Low-resolution Disease Maps. *International Journal of Health Geographics* 5, 1 (2006), 56.
- [12] Noppawan Buamithup, Kannikar Intawong, and Veerasak Punyapornwithaya. 2025. Geographical Distribution, Spatial Directional Trends, and Spatio-Temporal Clusters of the First Rapid and Widespread Lumpy Skin Disease Outbreaks in Thailand. *Transboundary and Emerging Diseases* 2025, 1 (2025), 4900775.
- [13] Christopher A Cassa, Shaun J Grannis, J Marc Overhage, and Kenneth D Mandl. 2006. A context-sensitive approach to anonymizing spatial surveillance data: impact on outbreak detection. *Journal of the American Medical Informatics Association* 13, 2 (2006), 160–165.
- [14] Christopher A Cassa, Shannon C Wieland, and Kenneth D Mandl. 2008. Re-identification of home addresses from spatial locations anonymized by Gaussian skew. *International journal of health geographics* 7, 1 (2008), 45.
- [15] Centers for Disease Control and Prevention. 2012. *Cartographic Guidelines for Public Health*. Technical Report. Centers for Disease Control and Prevention.
- [16] A. Chandran and P. Roy. 2024. Applications of geographical information system and spatial analysis in Indian health research: a systematic review. *BMC Health Services Research* 24 (2024), 1448.
- [17] Alex Clark and Pillow Contributors. 2024. Pillow: The Friendly PIL Fork. <https://pillow.readthedocs.io/>.
- [18] L. P. Clark, D. Zilber, C. Schmitt, et al. 2025. A review of geospatial exposure models and approaches for health data integration. *Journal of Exposure Science & Environmental Epidemiology* 35 (2025), 131–148.
- [19] OpenStreetMap contributors. 2024. OpenStreetMap. <https://www.openstreetmap.org>.
- [20] Franklin C Crow. 1977. The aliasing problem in computer-generated shaded images. *Commun. ACM* 20, 11 (1977), 799–805.
- [21] Andrew Curtis, Jacqueline Mills, and Michael Leitner. 2006. Keeping an eye on privacy issues with geospatial data. *Nature* 441, 7090 (2006), 150.
- [22] Andrew J. Curtis, J. W. Mills, and Michael Leitner. 2006. Spatial confidentiality and GIS: re-engineering mortality locations from published maps about Hurricane Katrina. *International Journal of Health Geographics* 5, 1 (2006), 44.
- [23] Anna Dmowska and Tomasz F Stepinski. 2019. Racial dot maps based on dasy-metrically modeled gridded population data. *Social Sciences* 8, 5 (2019), 157.
- [24] Edoardo Alberto Dominici, Nico Schertler, Jonathan Griffin, Shayan Hoshiyari, Leonid Sigal, and Alla Sheffer. 2020. PolyFit: Perception-Aligned Vectorization of Raster Clip-Art via Intermediate Polygonal Fitting. *ACM Transactions on Graphics* 39, 4 (2020), 77:1–77:16.
- [25] Cynthia Dwork. 2006. Differential Privacy. In *Automata, Languages and Programming*. 1–12.
- [26] Natalia Dziuba and Stanisław Szombara. 2025. Supporting the editing of dot maps using the spectral clustering algorithm. *Polish Cartographical Review* 57, 1 (2025), 58–74.
- [27] John Eck, Spencer Chainey, James Cameron, and Ronald Wilson. 2005. Mapping crime: Understanding hotspots.
- [28] Khaled El Emam, Fida Kamal Dankar, Romeo Issa, Elizabeth Jonker, Daniel Amyot, Elise Cogo, Jean-Pierre Corriveau, Mark Walker, Sadrul Chowdhury, Regis Vaillancourt, et al. 2009. A globally optimal k-anonymity method for the de-identification of health data. *Journal of the American Medical Informatics Association* 16, 5 (2009), 670–682.
- [29] Esri. [n. d.]. ArcGIS. <https://www.arcgis.com>.
- [30] Esri. 2016. World Imagery. <https://www.arcgis.com/home/item.html?id=10df2279f684e4a9f6a7f08febac2a9>.
- [31] Esri. 2022. Esri World Topographic Map. <https://www.arcgis.com/home/item.html?id=6e850093c837475e8c23d905ac43b7d0>.
- [32] European Union. 2016. Regulation (EU) 2016/679 of the European Parliament and of the Council. *Official Journal of the European Union* L119 (2016), 1–88.
- [33] Randi Foraker, Joshua Landman, Ian Lackey, Matthew D Haslam, Alison L Antes, and Dennis Goldfarb. 2022. Enabling hotspot detection and public health response to the COVID-19 pandemic. *Preventing chronic disease* 19 (2022), E35.
- [34] Herbert Freeman. 1974. Computer processing of line-drawing images. *ACM Computing Surveys (CSUR)* 6, 1 (1974), 57–97.
- [35] GeoPandas Developers. 2024. GeoPandas: Python tools for geographic data. <https://geopandas.org/>.
- [36] Google Developers. 2026. Geolocation API overview. <https://developers.google.com/maps>.
- [37] Cindy M Goral, Kenneth E Torrance, Donald P Greenberg, and Bennett Battaile. 1984. Modeling the interaction of light between diffuse surfaces. *ACM SIGGRAPH computer graphics* 18, 3 (1984), 213–222.
- [38] hailu kebede, Mohammed Motuma Assen, and Merid Abadi Sharew. 2026. Crime hotspot analysis and mapping using geospatial technology in Dessie city, Ethiopia. *Next Research* 5 (2026), 101303.
- [39] Shayan Hoshiyari, Edoardo Alberto Dominici, Alla Sheffer, Nathan Carr, Duygu Chaylan, Zhaoen Wang, and I-Chao Shen. 2018. Perception-Driven Semi-Structured Boundary Vectorization. *ACM Transactions on Graphics* 37, 4 (2018), 118:1–118:14.
- [40] Michael A Jochim. 2023. Dots on the map: Issues in the archaeological analysis of site locations. *Journal of Archaeological Method and Theory* 30, 3 (2023), 876–894.
- [41] Christoph Keller. 2023. Distribution of Badorf and Walberberg Ware in the British Isles during the 8th and 9th centuries. <https://zenodo.org/records/10013458>.
- [42] Victor Kesten. 2017. Evaluating Different Spatial Anti Aliasing Techniques.
- [43] Nikola Koktáva and Jiří Horák. 2023. Options for micro-mobility data visualization. *European Journal of Geography* 14, 4 (2023), 46–52.
- [44] Johannes Kopf and Dani Lischinski. 2011. Depixelizing Pixel Art. In *ACM SIGGRAPH 2011 Papers*. 99:1–99:8.
- [45] Olga Kounadi and Michael Leitner. 2014. Why Does Geoprivacy Matter? The Scientific Publication of Confidential Data Presented on Maps. *Journal of Empirical Research on Human Research Ethics* 9, 4 (2014), 34–45.
- [46] Anders Krogh and John Hertz. 1991. A simple weight decay can improve generalization. *Advances in neural information processing systems* 4 (1991).
- [47] Harold Joseph Kushner and Dean S Clark. 2012. *Stochastic approximation methods for constrained and unconstrained systems*. Vol. 26.
- [48] Mei-Po Kwan, Irene Casas, and Ben Schmitz. 2004. Protection of geoprivacy and accuracy of spatial information: How effective are geographical masks? *Cartographica: The International Journal for Geographic Information and Geovisualization* 39, 2 (2004), 15–28.
- [49] Quoc V. Le, Jiquan Ngiam, Adam Coates, Abhik Lahiri, Bobby Prochnow, and Andrew Y. Ng. 2011. On optimization methods for deep learning. In *Proceedings of the 28th International Conference on International Conference on Machine Learning*. 265–272.
- [50] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *nature* 521, 7553 (2015), 436–444.
- [51] Kaley Leetaru, Shaowen Wang, Guofeng Cao, Anand Padmanabhan, and Eric Shook. 2013. Mapping the global Twitter heartbeat: The geography of Twitter. *First Monday* (2013).
- [52] Michael Leitner and Andrew Curtis. 2004. Cartographic guidelines for geographically masking the locations of confidential point data. *Cartographic Perspectives*

- 49 (2004), 22–39.
- [53] Moritz Leitner and Andrew Curtis. 2006. A first step towards a framework for presenting the location of confidential point data on maps—results of an empirical perceptual study. *International Journal of Geographical Information Science* 20, 7 (2006), 813–822.
- [54] Michael Leitner, Jacqueline W. Mills, and Andrew Curtis. 2007. Can Novices to Geospatial Technology Compromise Spatial Confidentiality? *KN - Journal of Cartography and Geographic Information* 57, 2 (2007), 78–84.
- [55] William J Leler. 1980. Human vision, anti-aliasing, and the cheap 4000 line display. *ACM Siggraph Computer Graphics* 14, 3 (1980), 308–313.
- [56] Ninghui Li, Wahbeh Qardaji, Dong Su, Yi Wu, and Weining Yang. 2013. Membership privacy: A unifying framework for privacy definitions. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 889–900.
- [57] Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Jonathan Ragan-Kelley. 2020. Differentiable Vector Graphics Rasterization for Editing and Learning. *ACM Transactions on Graphics* 39, 6 (2020), 1–15.
- [58] Hsien-Ho Lin, Seungyun Shin, Joaquin Blaya, Z Zhang, Peter Cegielski, Carmen Contreras, L Asencios, Cristian Bonilla, Juliana Bayona, C Paciorek, and Ted Cohen. 2010. Assessing spatiotemporal patterns of multidrug-resistant and drug-sensitive tuberculosis in a South American setting. *Epidemiology and infection* 139 (12 2010), 1784–93.
- [59] Timothy Lottes. 2009. A quick overview of MSAA. https://developer.download.nvidia.com/assets/gamedev/files/sdk/11/FXAA_WhitePaper.pdf.
- [60] Yongmei Lu, Charles Yorke, and F Benjamin Zhan. 2012. Considering risk locations when defining perturbation zones for geomasking. *Cartographica: The International Journal for Geographic Information and Geovisualization* 47, 3 (2012), 168–178.
- [61] B. F. Martinez, J. L. Annett, E. M. Kilbourne, M. L. Kirk, K. Lui, and S. M. Smith. 1989. Geographic Distribution of Heat-Related Deaths Among Elderly Persons: Use of County-Level Dot Maps for Injury Surveillance and Epidemiologic Research. *JAMA* 262, 16 (1989), 2246–2250.
- [62] R. K. Matiaschuk, I. V. Goncharenko, I. V. Tkachenko, Y. S. Prokopyuk, and K. Y. Shehur. 2015. Taxonomic composition and spatial structure of the elements of the Feofaniya park. *Ecology and Noospherology* 26, 3–4 (2015), 21–29.
- [63] Kusnul Yuli Maulana, Kannika Na-Lampang, Orapun Arjkumpa, Noppawan Buamithup, Kannikar Intawong, and Veerasak Punyapornwithaya. 2025. Geographical Distribution, Spatial Directional Trends, and Spatio-Temporal Clusters of the First Rapid and Widespread Lumpy Skin Disease Outbreaks in Thailand. *Transboundary and Emerging Diseases* 2025, 1 (2025), 4900775.
- [64] James B McQueen. 1967. Some methods of classification and analysis of multivariate observations. In *Proc. of 5th Berkeley Symposium on Math. Stat. and Prob.* 281–297.
- [65] Cristian S Montalvo-Mancheno, Jessie C Buettel, Stefania Onde, and Barry W Brook. 2025. A Reproducible, Data-Driven Approach to Mapping Species Distributions Using Presence-Only Data and Biogeographic Templates. *Ecology and Evolution* 15, 10 (2025), e72285.
- [66] Abdulkader Murad and Bandar Fuad Khashoggi. 2020. Using GIS for disease mapping and clustering in Jeddah, Saudi Arabia. *ISPRS International Journal of Geo-Information* 9, 5 (2020), 328.
- [67] III Obaldia, Nicanor. 2015. Determinants of low socio-economic status and risk of Plasmodium vivax malaria infection in Panama (2009–2012): a case-control study. *Malaria Journal* 14 (2015), 14.
- [68] M Papeş and Philippe Gaubert. 2007. Modelling ecological niches from low numbers of occurrences: assessment of the conservation status of poorly known viverrids (Mammalia, Carnivora) across two continents. *Diversity and distributions* 13, 6 (2007), 890–902.
- [69] Matt Pettineo. 2012. A quick overview of MSAA. <https://mynameismjp.wordpress.com/2012/10/24/msaa-overview/>.
- [70] QGIS Development Team. 2024. QGIS: A Free and Open Source Geographic Information System. <https://qgis.org/>.
- [71] Brian D Ripley et al. 2001. The R project in statistical computing. *MSOR connections. The newsletter of the LTSN Maths, Stats & OR Network* 1, 1 (2001), 23–25.
- [72] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. 1986. Learning representations by back-propagating errors. *nature* 323, 6088 (1986), 533–536.
- [73] Sapphire Nation. 2016. Anti-aliasing techniques comparison. <https://www.sapphirenation.net/anti-aliasing-comparison-performance-quality>.
- [74] Reza Shokri, George Theodorakopoulos, Jean-Yves Le Boudec, and Jean-Pierre Hubaux. 2011. Quantifying location privacy. In *2011 IEEE symposium on security and privacy*. IEEE, 247–262.
- [75] Catherine Smith, Steven Le Comber, Hannah Fry, Matthew Bull, Steve Leach, and Ac Hayward. 2015. Spatial methods for infectious disease outbreak investigations: Systematic literature review. *Eurosurveillance* 20 (2015).
- [76] Duncan Smith. 2020. World Population Density. <https://luminocity3d.org/WorldPopDen/>.
- [77] Laura Soetens, Susan Hahné, and Jacco Wallinga. 2017. Dot map cartograms for detection of infectious disease outbreaks: an application to Q fever, the Netherlands and pertussis, Germany. *Eurosurveillance* 22, 26 (2017), 30562.
- [78] James C Spall. 2002. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE transactions on automatic control* 37, 3 (2002), 332–341.
- [79] State of California Legislature. 2018. California Consumer Privacy Act of 2018. *California Civil Code* (2018), 1798.
- [80] Dave Stinchcomb. 2004. Procedures for geomasking to protect patient confidentiality. In *ESRI international health GIS conference*. 17–20.
- [81] Latanya Sweeney. 2002. k-anonymity: A model for protecting privacy. *International journal of uncertainty, fuzziness and knowledge-based systems* 10, 05 (2002), 557–570.
- [82] Tableau Software, LLC. 2024. Tableau: Business Intelligence and Analytics Software. <https://www.tableau.com>.
- [83] Simon Teufel, Annika Beinflich, Michael Bender, Tobias Weber, Daniel Doerr, Holger Winnemöller, and Andreas Kolb. 2024. A Survey of Smooth Vector Graphics: Recent Advances in Representation, Creation, Rasterization and Image Vectorization. *IEEE Transactions on Visualization and Computer Graphics* 30, 3 (2024), 1652–1671.
- [84] Marceline A. Tewara, Paul N. Mbah-Fongkimeh, Abdurahman Dayimu, et al. 2018. Small-area spatial statistical analysis of malaria clusters and hotspots in Cameroon; 2000–2015. *BMC Infectious Diseases* 18, 1 (2018), 636.
- [85] Markus Viljanen, Lisa Tostrams, Niels Schoffelen, Jan van de Kasstelee, Leon Marshall, Merijn Moens, Wouter Beukema, and Wieger Wamelink. 2024. A joint model for the estimation of species distributions and environmental characteristics from point-referenced data. *Plos one* 19, 6 (2024), e0304942.
- [86] Shannon C Wieland, Christopher A Cassa, Kenneth D Mandl, and Bonnie Berger. 2008. Revealing the spatial distribution of a disease while preserving privacy. *Proceedings of the National Academy of Sciences* 105, 46 (2008), 17608–17613.
- [87] Wikipedia. 2025. 1854 Broad Street cholera outbreak. https://en.wikipedia.org/wiki/1854_Broad_Street_cholera_outbreak.
- [88] Jinfan Yang, Nicholas Vining, Shakiba Kheradmand, Nathan Carr, Leonid Sigal, and Alla Sheffer. 2023. Subpixel Deblurring of Anti-Aliased Raster Clip-Art. *Computer Graphics Forum* 42, 2 (2023), 61–76.
- [89] Xiao Yuan. 2020. The Application of Geographic Information System (GIS) in Academic Success Center (ASC) of a Medium-Sized Liberal Art University. *Educational Research: Theory and Practice* 31, 3 (2020), 94–100.
- [90] Paul A. Zandbergen. 2014. Ensuring Confidentiality of Geocoded Health Data: Assessing Geographic Masking Strategies for Individual-Level Data. *Advances in Medicine* 2014, 1 (2014), 567049.
- [91] Dale L Zimmerman and Claire Pavlik. 2008. Quantifying the effects of mask metadata disclosure and multiple releases on the confidentiality of geographically masked health data. *Geographical analysis* 40, 1 (2008), 52–76.

A Open Science

Our artifact includes (i) source code for proposed attacks, (ii) the benchmarks, and (iii) the privacy assessment tool. The repository is available at <https://github.com/PuddlesPenguin/AutoLocate/>.

B Ethics Considerations

Our research investigates the privacy risks associated with dot maps, specifically focusing on high-precision location recovery from rasterized maps. Since dot maps are widely used to visualize sensitive data (e.g., patient home addresses and crime locations), we recognize our responsibility to carefully assess the ethical implications of our findings. We have undertaken this assessment using the framework outlined in the Menlo Report, while adhering to the ethical guidelines set forth by CCS 2026.

Stakeholder-Based Analysis. This research involves several key stakeholders, each impacted by our findings in different ways:

- **Researchers and Map Creators.** Our primary audience consists of researchers and creators of dot maps. We provide these practitioners with a deeper understanding of the privacy risks in dot maps, along with a concrete tool for assessing the risks of their own maps. Additionally, we propose and validate mitigation strategies to address these risks.
- **Data Subjects.** The data subjects in this context are the individuals whose sensitive location data is visualized on dot maps. In this paper, our experiments were conducted using synthetic/public

datasets, and no specific individuals or proprietary dot maps were targeted. Furthermore, we believe it is important to raise awareness about these underlying privacy risks and prevent potential privacy threats to individuals in the future.

- **Map Software Developers.** The developers of map visualization platforms (e.g., QGIS, GeoPandas, and R) are also stakeholders in this research, as our attack exploits a default rendering feature (i.e., anti-aliasing) present in these platforms. By publishing this work, we aim to provide developers with insights to incorporate techniques that can mitigate such privacy risks.
- **Adversaries.** Our methods could be maliciously used by adversaries to identify individuals or specific locations from dot maps. However, it is important to note that these risks already existed prior to our research. We believe that by raising awareness of these risks, we can help mitigate broader privacy concerns. Additionally, we discuss effective mitigation strategies to minimize the likelihood of malicious use of this research.

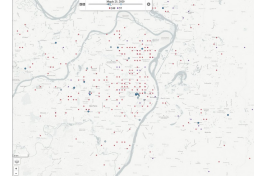
Ethical Justification. Dot maps are commonly used in sensitive domains, where the potential risks of exposing individuals' locations are significant. Given that these risks are not always well understood, we believe it is crucial to disclose the vulnerabilities associated with publishing dot maps. While we recognize that malicious actors could exploit our findings, we believe that proactively sharing this knowledge enables the research community to address these privacy risks before they are exploited in real-world scenarios. We encourage the community to use our results to develop stronger privacy protections.

Responsible Disclosure. We have disclosed our findings to the developers of the map visualization platforms evaluated in this paper, including QGIS (through its security team), Matplotlib (which GeoPandas relies on for rendering), and the maintainer of the R maps package. We have also reported our findings to the U.S. CDC, which publishes cartographic guidelines [15] for sensitive data. Several of these stakeholders have engaged with us, and we are working with them to address this risk.

C Literature Survey of Dot Map Usage

To demonstrate the widespread prevalence of dot maps, we conducted a broad survey covering both academic research and public media. Our methodology utilized academic databases (Google Scholar, Semantic Scholar) for research publications and general search engines for journalistic and governmental examples. We combined general terms for the visualization technique (e.g., “dot map”, “point map”, “spatial distribution”) with domain-specific keywords. The queries for each category were structured as follows:

- For **Public Health**, we used (“public health” OR “epidemiology”) AND “dot map” AND (“patient location” OR “case distribution”).
- For **Criminology**, we used “crime mapping” AND “point map” AND “incident location”.
- For **Ecology**, we used “ecology” AND “point map” AND (“endangered species” OR “presence-only data”).
- For **Social Science**, we used (“social science” OR “demography”) AND “dot density map” AND (“household demographics” OR “racial dot map”).
- For **Education**, we used “education” AND “spatial analysis” AND “student residence”.



(a) Excerpted from Figure 1 of [33].



(b) Excerpted from Figure 1 of [67].

Figure 7: Examples of dot maps in published papers, where most dots are isolated.

Algorithm 3 FindBoundaryPixels. It identifies the boundary pixels of each dot from the target dot map.

Require: Dot regions $\mathcal{P}$, estimated dot locations $C = \{c_1, \dots, c_n\}$

```

1:  $\mathcal{S}_i \leftarrow \emptyset$  for  $i = 1, \dots, n$ 
2: for each component  $P \in \mathcal{P}$  do
3:   for each pixel  $(x, y) \in P$  do
4:     for each four-neighbor  $(u, v)$  of  $(x, y)$  do
5:       if  $(u, v) \notin P$  then
6:          $i \leftarrow \arg \min_j \|(u, v) - c_j\|$  ▷ assign to the nearest dot
7:          $\mathcal{S}_i \leftarrow \mathcal{S}_i \cup \{(u, v)\}$ 
8:       end if
9:     end for
10:  end for
11: end for
12: return  $\{\mathcal{S}_i\}_{i=1}^n$ 
```

- For **Archaeology**, we used “archaeology” AND “point map” AND “artifact find spot”.

For every result, we manually inspected the source (whether an academic paper, news article, or web report) to confirm it utilized a raster dot map where each dot represents a discrete data point (e.g., one person or incident). If a source was not a direct match, we examined its citations or references to identify related examples. The resulting selected publications are shown in Table 1. Note that this is an exemplary list intended to showcase the prevalence of dot map usage, not a comprehensive or systematic review. We refer the reader to [75] for a more complete survey of spatial visualization usage in academic research.

Isolated Dots in Published Maps. We examine real published dot maps to assess whether our attack is applicable in practice. Specifically, we manually count the dots in two representative maps from Table 1, treating a dot as isolated if it does not overlap with any other dot. As shown in Figure 7, isolated dots account for the majority of dots in both maps. The map from [33] contains 306 dots, of which 287 (93.8%) are isolated, while 6 out of the 9 (red) dots (66.7%) in a map from [67] are isolated. AutoLocate achieves strong location recovery performance on isolated dots; their prevalence in published maps indicates that our attack is applicable in practice.

D Implementation Details

Implementations of AutoLocate. By default, AutoLocate renders one candidate map for each search direction, in which all dots are shifted simultaneously, which we refer to as the *batched*

Table 20: Performance comparison of the batched and per-dot implementations of AutoLocate.

Dataset	Method	Implementation	Dist. Error (m) by # Overlapping Dots					Runtime (min)
			1	2	3	4	5	
OpenAddresses	AutoLocate _{BU}	Batched	1.81 ±.66	7.34 ±102.29	8.89 ±185.73	27.00 ±1046.78	16.12 ±11828.85	3.58
		Per-dot	1.42 ±.57	7.41 ±111.63	8.79 ±162.48	22.00 ±987.31	15.70 ±12417.52	87.7
	AutoLocate _{BK}	Batched	0.95 ±.64	2.26 ±1.04	4.31 ±32.54	4.86 ±241.87	5.46 ±10264.44	3.76
		Per-dot	0.85 ±.68	2.34 ±.95	3.31 ±35.82	4.74 ±207.41	4.95 ±11203.76	73.3
Synthetic	AutoLocate _{BU}	Batched	1.69 ±.51	3.00 ±1.66	4.79 ±50.39	4.39 ±161.92	5.37 ±641.73	3.58
		Per-dot	1.61 ±.46	3.02 ±1.81	4.56 ±43.27	4.52 ±178.64	5.66 ±587.19	137.3
	AutoLocate _{BK}	Batched	0.88 ±.52	1.82 ±.70	2.17 ±.99	2.50 ±15.54	2.92 ±1158.51	3.76
		Per-dot	0.76 ±.55	1.86 ±.64	1.95 ±1.09	2.53 ±13.91	3.83 ±1274.83	145.1

Table 21: Map configuration space. The default setting for each dimension is marked in bold.

Dimension	Values	# Settings
Map scale	Small (US) , Medium (OH), Large (Austin, TX)	3
Background	White canvas, Street map , Satellite imagery	3
Resolution	96, 192 , 384 DPI	3
Dot geometry	Circle , Pentagon, Triangle	3
Dot size	1 mm, 2 mm , 3 mm	3
Format	PNG , JPEG, TIFF	3
Platform	GeoPandas , QGIS, R	3

Table 22: Mitigation strategies against AutoLocate_{BU}.

Mitigation	OpenAddresses		Synthetic	
	Dist. Error (m)	Rel. Px. Error	Dist. Error (m)	Rel. Px. Error
w/o Anti-aliasing	235.98 ±181.42	0.0874 ±.0715	232.31 ±120.83	0.0860 ±.0476
Geo-masking (radius: 100 m)	100.97 ±.97	0.0374 ±.0004	102.15 ±.87	0.0378 ±.0003
Geo-masking (radius: 50 m)	50.72 ±.85	0.0188 ±.0003	50.34 ±.86	0.0186 ±.0003
Quantization (3 Decimals)	38.27 ±12.22	0.0142 ±.0048	36.64 ±12.76	0.0136 ±.0050
Quantization (2 Decimals)	407.37 ±141.01	0.1509 ±.0555	396.97 ±123.69	0.1470 ±.0487
None	1.81 ±.66	0.0007 ±.0003	1.69 ±.51	0.0006 ±.0002

implementation. This design keeps the number of rendered maps

in each optimization iteration independent of the number of dots. An alternative is to render a separate candidate map for each dot and compute its loss individually, which we refer to as the *per-dot implementation*. We compare the two implementations in terms of recovery error and runtime. As shown in Table 20, the two implementations achieve comparable median recovery errors for both isolated and overlapping dots. The per-dot implementation yields only modest improvements in median error in a few cases (e.g., from 27.00 m to 22.00 m for four overlapping dots on OpenAddresses using AutoLocate_{BU}). In contrast, it requires over an order of magnitude more runtime. We therefore adopt the batched implementation for its efficiency.

Boundary Pixel Identification. Algorithm 3 presents the procedure for identifying the boundary pixels of each dot. It takes as input the dot regions $\mathcal{P}$ and the estimated dot locations $\mathcal{C}$ obtained in Phase 1 of Algorithm 2, and returns the boundary pixel set $\mathcal{S}_i$ for each dot c_i . For each dot region, the algorithm examines the four neighboring pixels of every pixel in the region and collects those that fall outside the region. Each collected pixel is then assigned to its nearest dot, forming the boundary pixel set for that dot.